

CÁC CÔNG NGHỆ LẬP TRÌNH HIỆN ĐẠI

XỬ LÝ GIAO DỊCH PHÂN TÁN

TS. Đỗ Như Tài
Đại Học Sài Gòn
dntai@sgu.edu.vn
1

Microservices Distributed Transactions

Saga Pattern for Distributed Transactions - Choreography and Orchestration

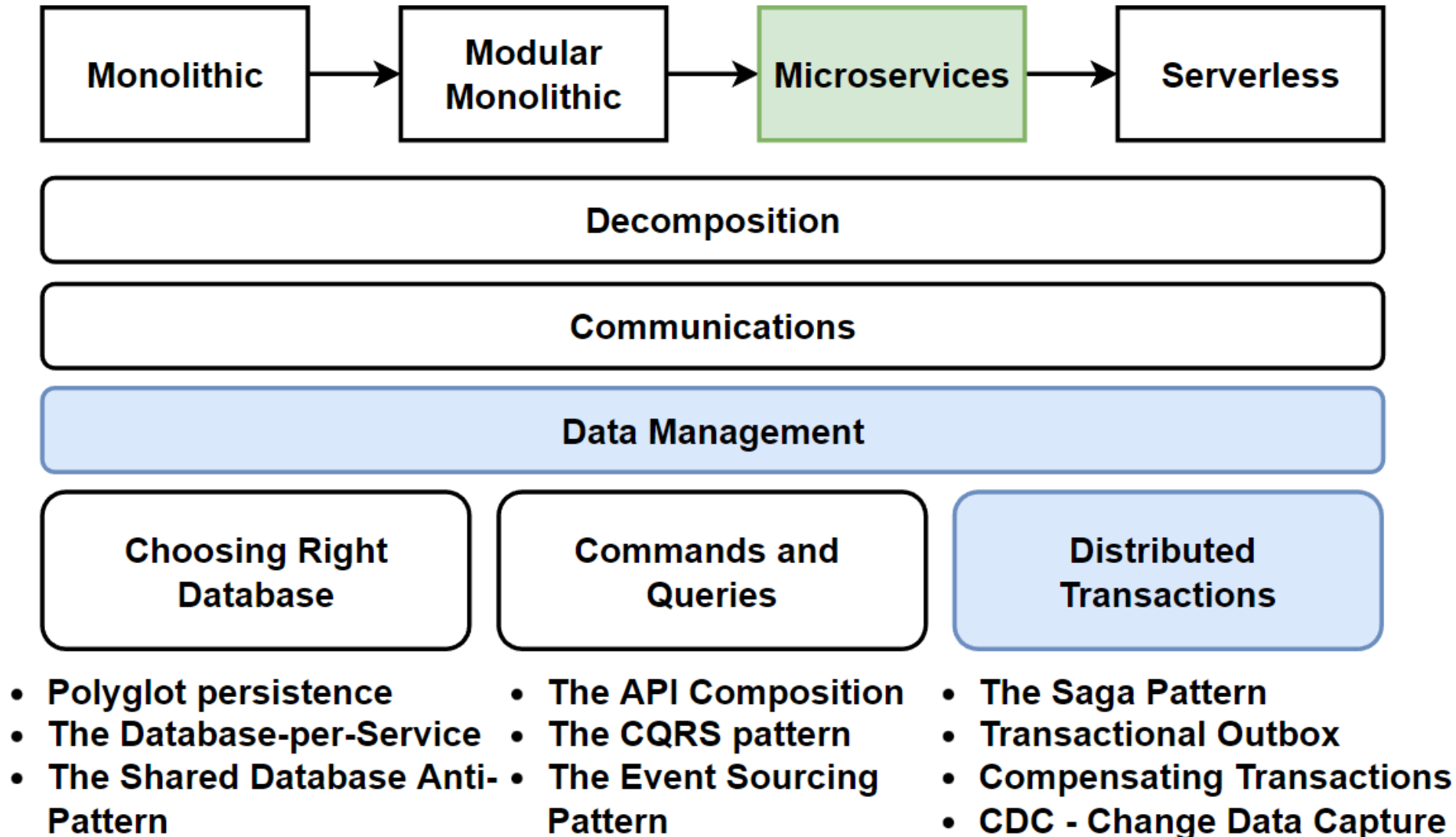
Transactional Outbox Pattern

Compensating Transaction pattern

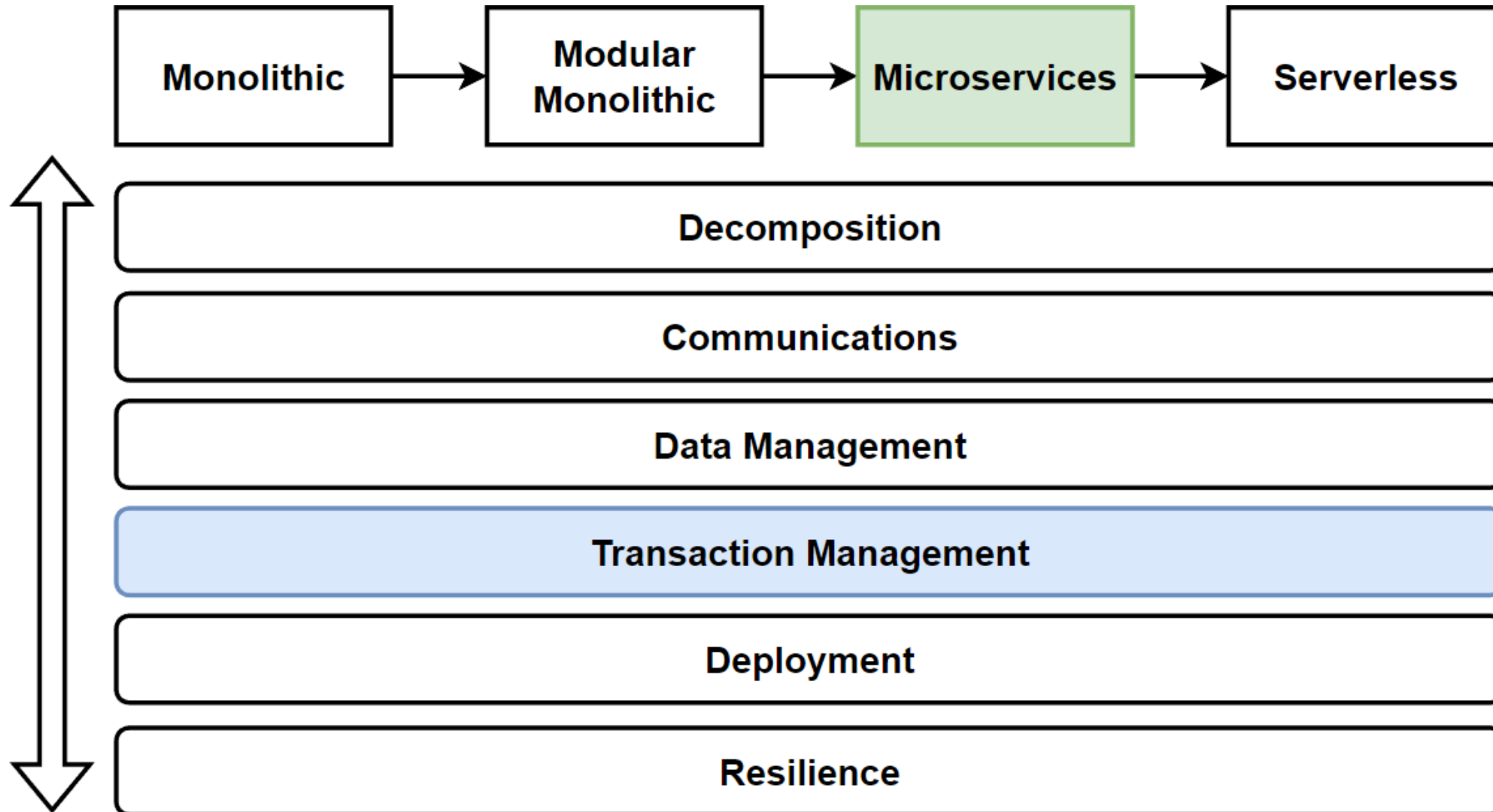
Dual-Write Problem

CDC - Change Data Capture

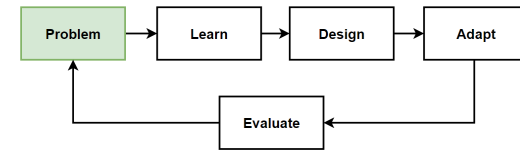
Microservices Data Management - Main Topics



Architecture Design – Vertical Considerations



Problem: Manage Consistency Across Microservices in Distributed Transactions



Considerations

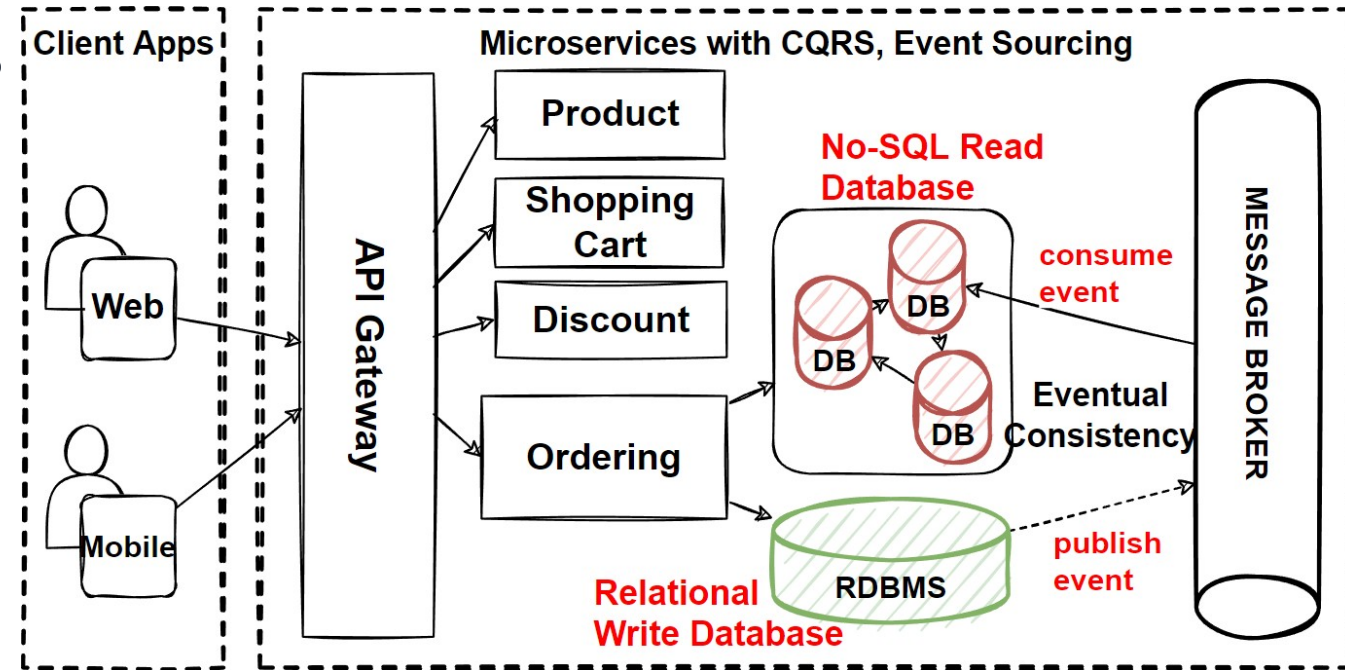
- Distributed Transactions that required to visit several microservices ?
- Consistency across multiple microservices ?
- Rollback transaction and run compensating steps ?

Problems

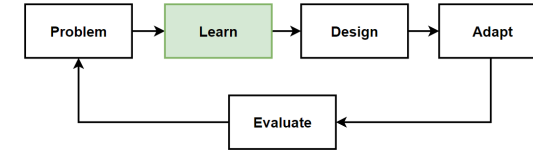
- Distributed Transaction Management
- Rollback Transaction on Distributed Environment
- Run Compensate Steps if one of service fail

Solutions

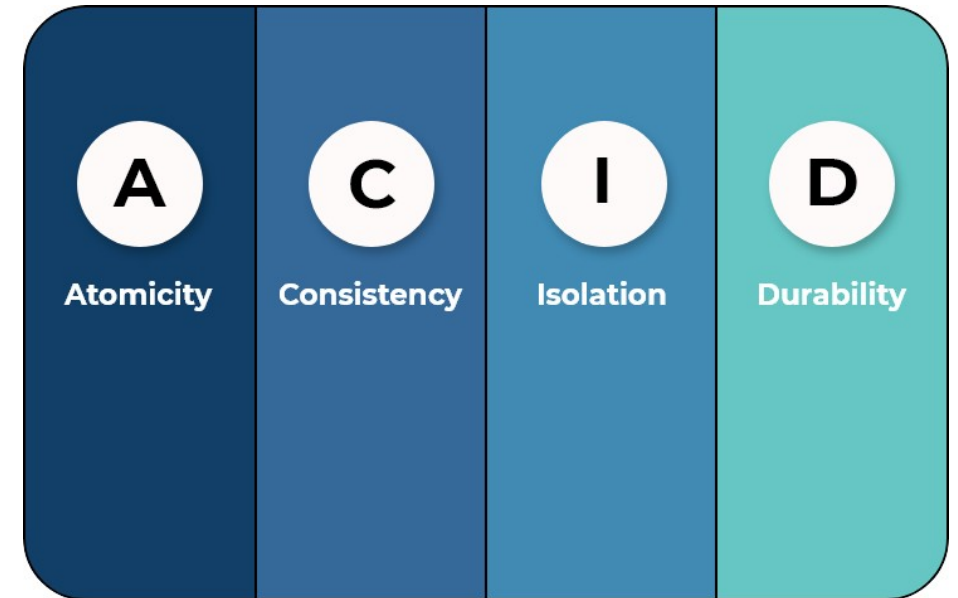
- Microservices Distributed Transaction Management Pattern and Best Practices
- Saga Pattern for Distributed Transactions
- Transactional Outbox Pattern
- Compensating Transaction pattern
- CDC - Change Data Capture



Transaction Fundamentals and ACID Principles

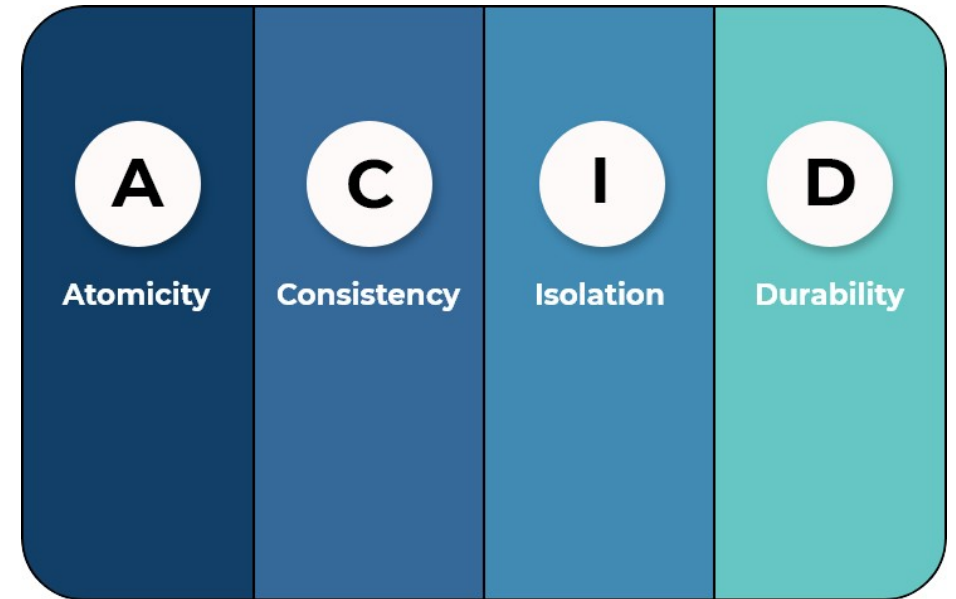
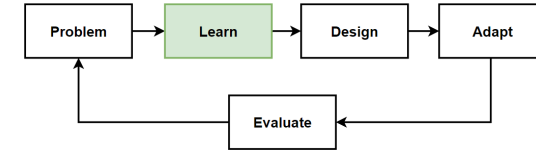


- **Transaction** is the **context of database operations** that should act to be **single unit of work**.
- Should **fully complete the operation** or if something fail, it should **rollback the whole operation** and go back to consistent state.
- **ACID - Atomicity, Consistency, Isolation, and Durability**
- **If one change fails** on database, the **whole transaction is going to fail**, and the database **should remain** in the **previous state**.
- **Relational Databases** are providing **ACID principles** to ensure Data Consistency.
- If you need **strong consistency**, its good to choose **Relational Databases**.

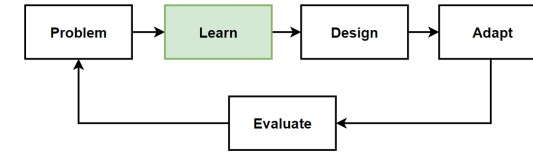


ACID Principles

- **ACID** - Atomicity, Consistency, Isolation, and Durability
- **Atomicity**
All operations are executed successfully or everything fails and rollback together.
- **Consistency**
The data in the database is kept in a valid and consistent state, after any operations on data.
- **Isolation**
Separated transactions running concurrently and they can not interfere with each other.
- **Durability**
After a transaction is committed, the changes are stored durably, for example persisted in a database server.
- **Example:** Money transfer transaction.



Transaction Management of Monolithic Architecture

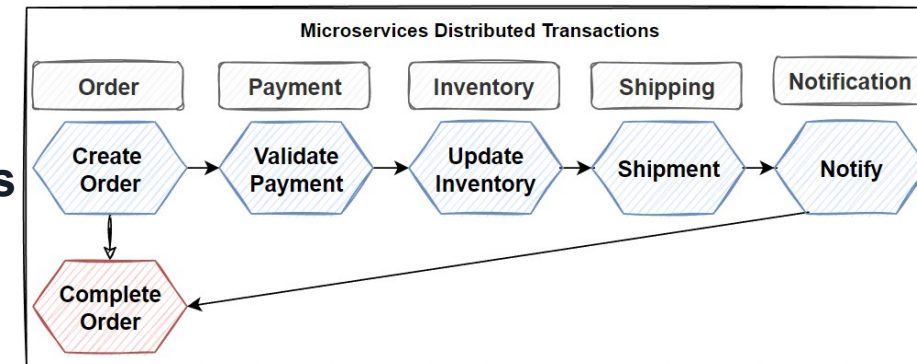
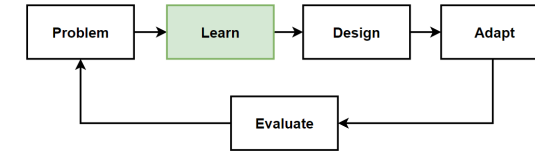


- **Transaction management** in Monolith architecture is quite easy compared to Microservice Architecture. Many frameworks contains mechanism for transaction management.
- These mechanism have a **single database** of the **whole application**. They are developed for scenarios where all transactions are running on a **single context**.
- Simply **commit** and **rollback** operations with these mechanism in monolith architectures.
- **Transactions operated** in the **transaction scope** are kept in memory without writing to the database until they are **committed**, and if a **Rollback** is made at any time, all transactions in the scope are **deleted** from memory and the **transaction** is **canceled**.
- When **Commit** is **written** to the **database**, the transaction is **completed successfully**.

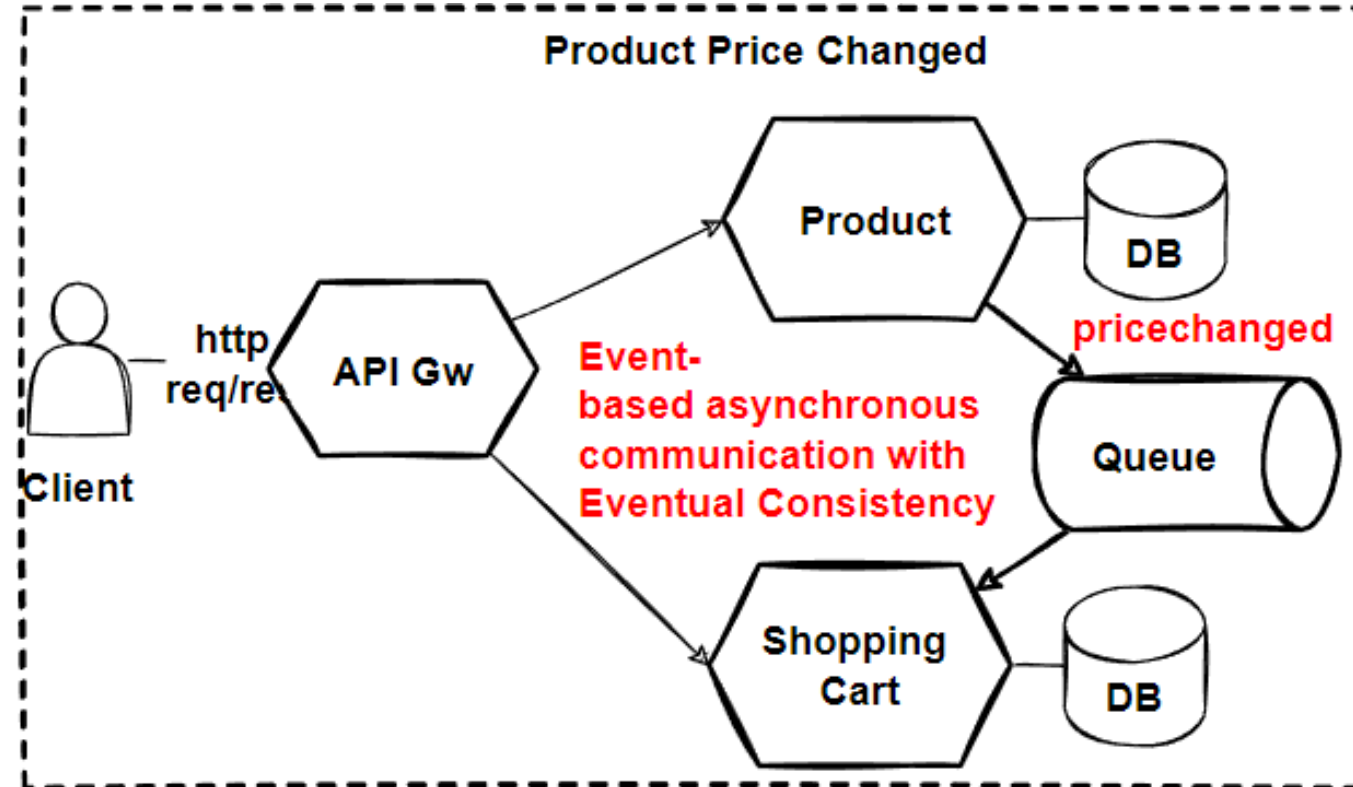
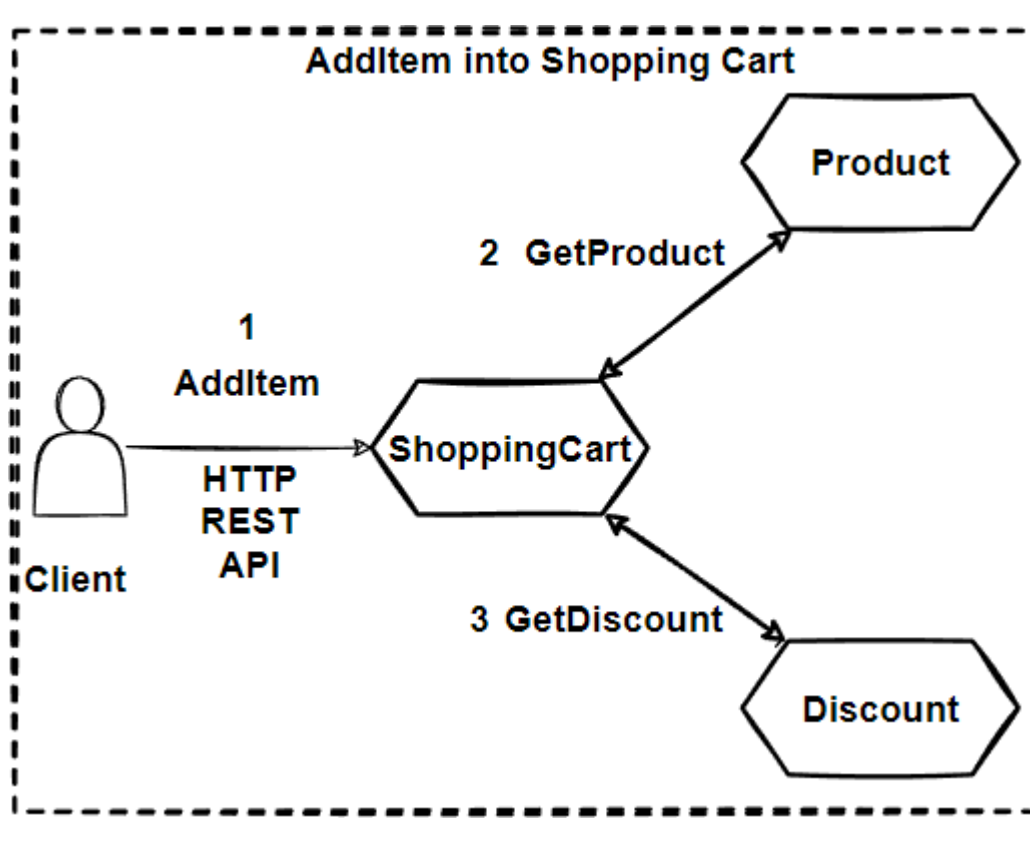
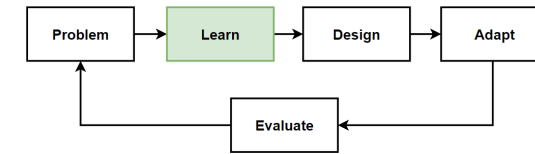
```
function place_order()  
  do_payment  
  decrease_stock  
  send_shipment  
  generate_bill  
  update_order
```


Microservices Distributed Transactions

- How to perform **transactional operations across microservices** and which patterns we should apply ?
- **Querying data** and **run complex join queries** across microservices is not easy but implement **transactional operations** across microservices is **more complex**.
- We should handle **distributed transaction managements** on microservices by manually implementing some **patterns** and **practices**
- **Distributed transaction** managements on microservices, mostly working with **eventual consistency**.
- Microservices has its own database and communicates each other with exposing APIs, it makes **harder to perform end-to-end business cases** across multiple microservices when trying to **keep data consistency**.
- How to **achieve consistency** across multiple microservices ?

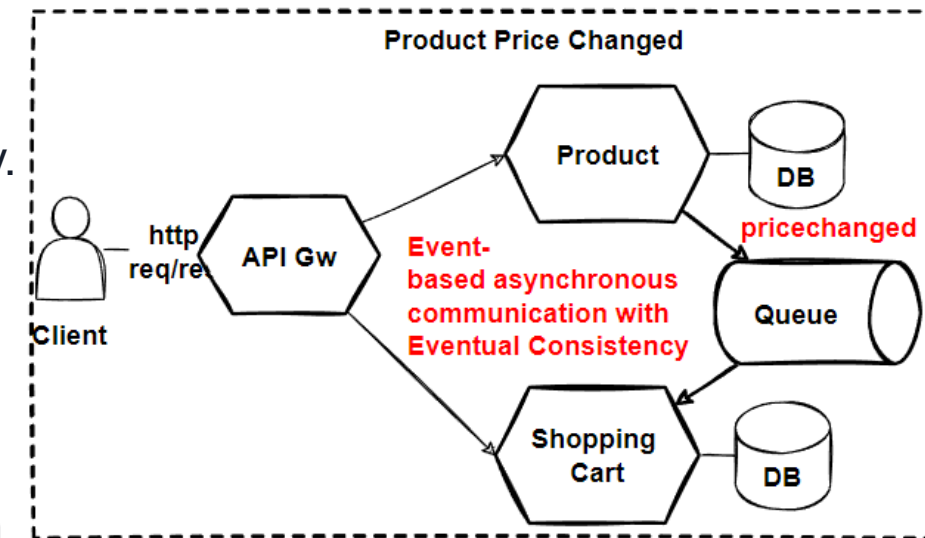
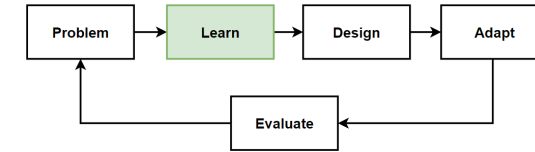


How to achieve Data Consistency across multiple microservices ?

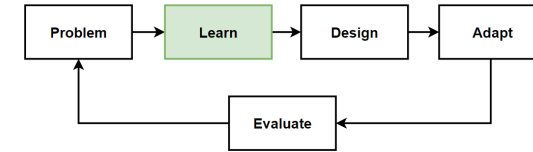


How to achieve Data Consistency across multiple microservices ? - 2

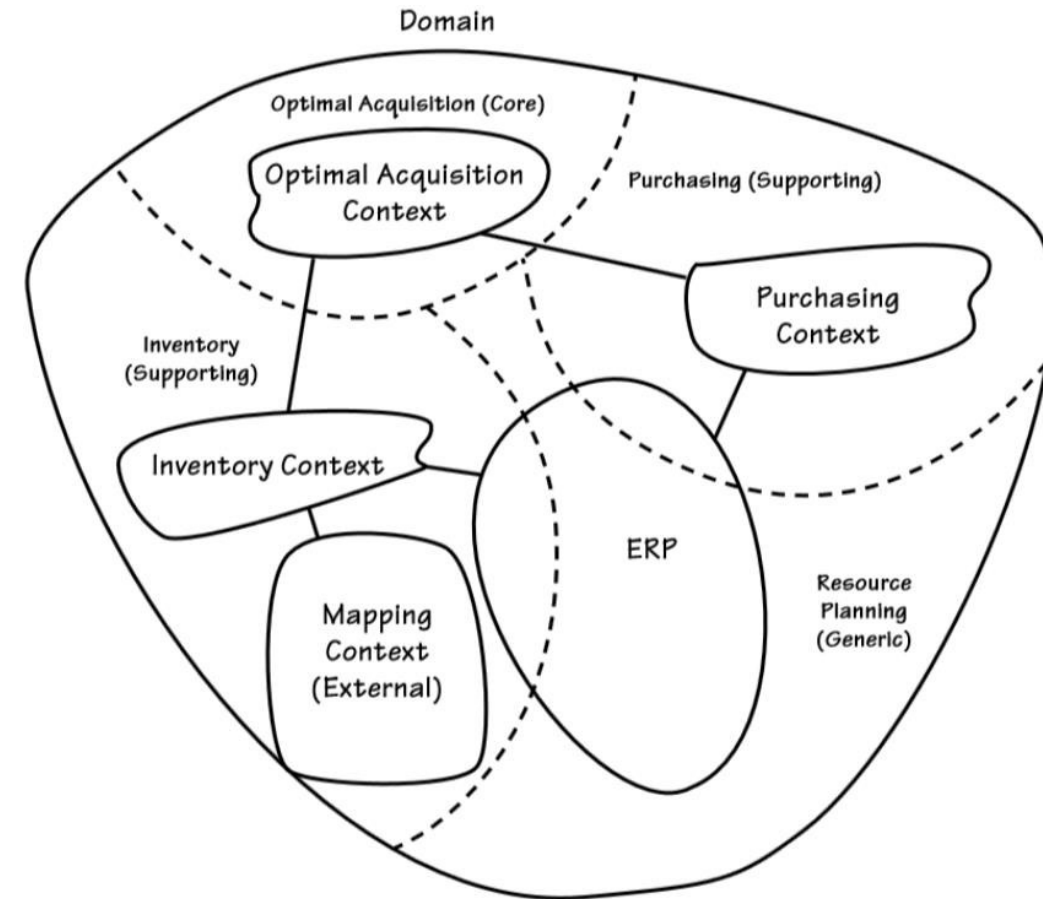
- **CAP Theorem:** need to **sacrifice ACID strong consistency** to the **High Availability** and **Partition Tolerance**.
- Most **microservice-based applications** are **choosing high availability** and **high scalability** against to strong consistency.
- Microservices **sacrifice strong consistency** and follow with **eventual consistency** to get benefits of microservices availability and scalability.
- **NoSQL database** are mostly using in microservices because they can easily and **horizontally scale** in distributed environments.
- Must use **asynchronous event-driven communication** and **publish/subscribe pattern** following to **eventual consistency**.
- When **product price updated**, Product service **publish changes** with Product price changed event to the Event Bus, that **subscribe by** ShoppingCart service with following eventual consistency principle.
- This process should be **resilient** and **idempotent** for redundant event processing; **Microservices Resilience**.



Bounded Context Pattern (Domain-Driven Design)

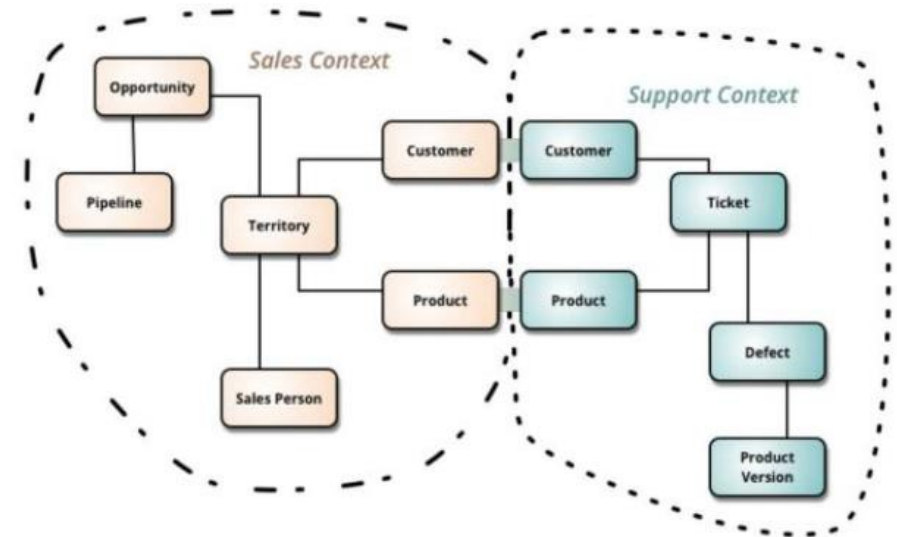
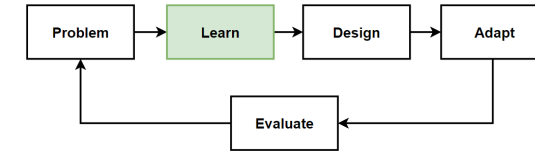


- **DDD - Bounded Context Pattern** which is one of the main pattern that we mainly use when **decomposing microservices**.
- Domains are **require high cooperation** and have a certain complexity by nature are called **collaborative domains**.
- DDD has 2 phases, Strategic and Tactical DDD.
- Strategic DDD, we define the large-scale model of the system, defining to the business rules that allow designing loosely coupling units and the context map between them.
- Tactical DDD focuses on implementation and provides design patterns that we can use to build the software implementation.
- Include concepts such as entity, aggregate, value object, repository, and domain service.



Bounded Context Pattern (Domain-Driven Design)-2

- **DDD** domain defines its **own common language** and divides boundaries into specific, independent components. Common language is called **ubiquitous language**, and independent units are called **bounded context**.
- **DDD** is solving a **complex** problem is to break the problem into **smaller parts** and focus on smaller problems that are relatively easy.
- **A complex domain** may contain **sub domains**. And some of sub domains can **combine** and **grouping** with each other for **common rules** and **responsibilities**.
- **Bounded Context** is the grouping of closely related scopes that we can say **logical boundaries**.
- **Bounded context** is the **logical boundary** that represents the smaller problem particles of the complex domain that **are self-consistent** and as **independent** as possible.

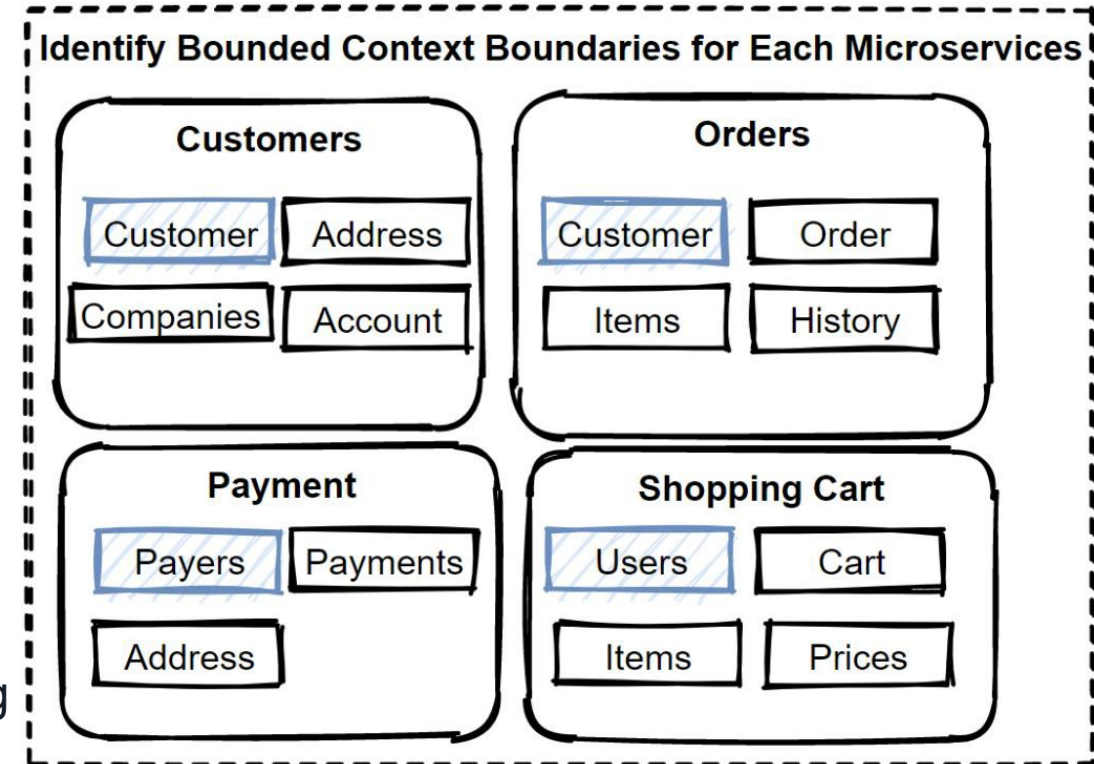
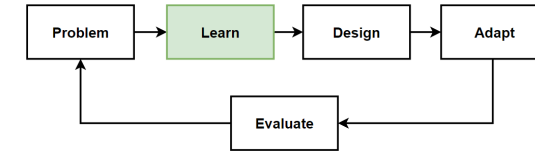


DDD deals with large models by dividing them into different Bounded Contexts and being explicit about their interrelationships.

<http://martinfowler.com/bliki/BoundedContext.html>

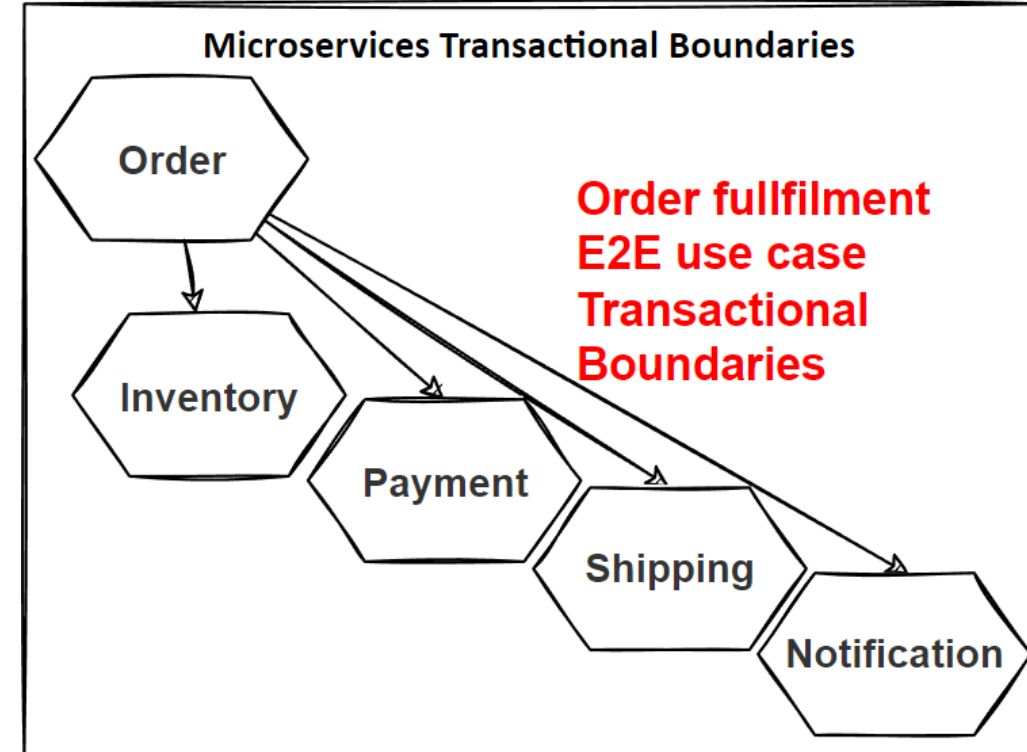
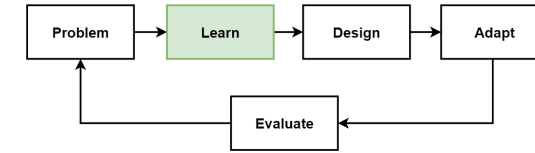
Identify Bounded Context Boundaries for Each Microservices

- **Identify** the **Bounded Contexts** by talking to the **domain experts** and using some clues.
- Once defined the Bounded Contexts, iterate design, those are **not immutable**.
- **Reshape** your **Bounded Contexts** by talking to the domain experts and consider **refactoring's** with the changing conditions.
- Its crucial to discuss with **domain experts** to defining **domains** and **sub domains**.
- **Evaluate Bounded Context** with the **domain experts** will help you identify to microservices.
- **Sub domains** inside of the **Bounded Context** are representing same data but naming differently due to domain experts areas.
- Should **discuss several domain experts** for their expertise areas.

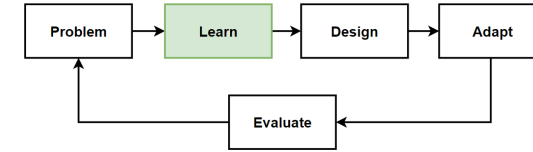


Microservices Transactional Boundaries

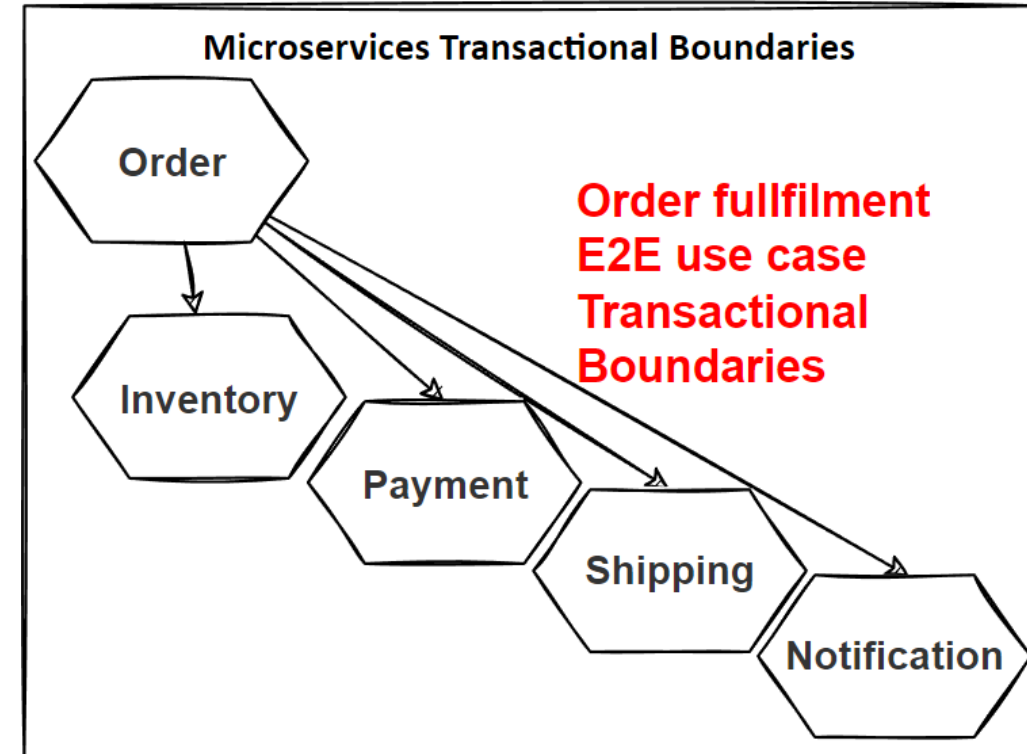
- We have several Bounded Context and Microservices for our e-commerce application:
- Customer-User, Product, Shopping Cart, Discount
- Ordering, Payment, Inventory, Shipment, Notification
- Some of these bounded context need to organize and communicate each other to perform End-to-end business use cases.
- After identifying Bounded Contexts and microservices, should also identify our microservices transactional boundaries.
- What are Microservices Transactional Boundaries ?



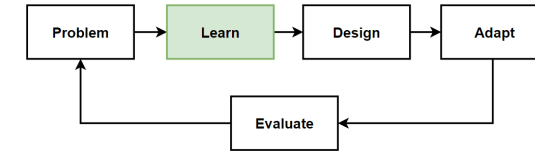
What are Microservices Transactional Boundaries ?



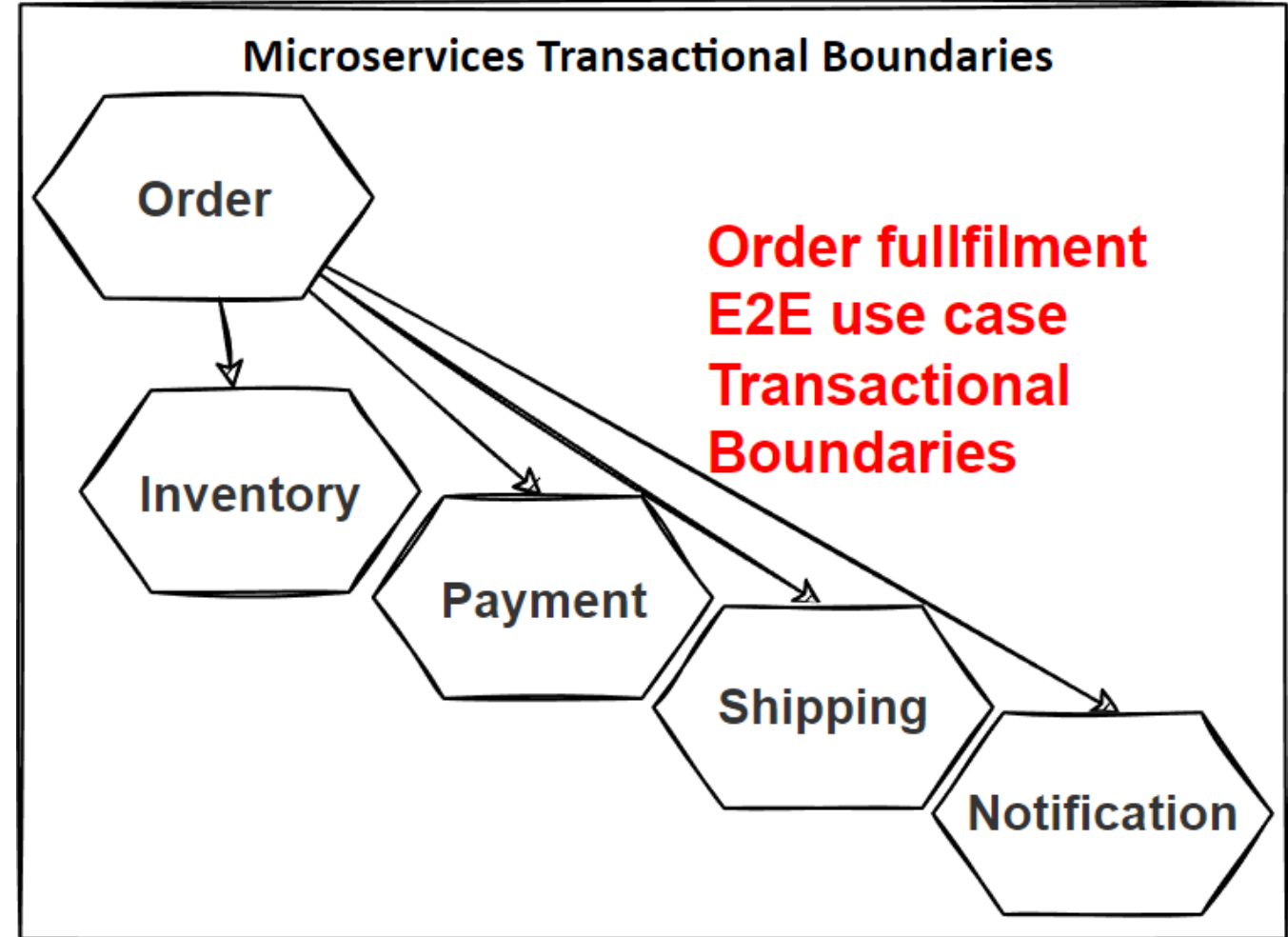
- **Transactional Boundaries** are **smallest unit of atomicity** that need to provide consistency between services.
- In **distributed architecture** we **can't provide ACID** principles.
- Should define **Transactional Boundaries** and perform **E2E use cases** to try to **keep data consistency**.
- Use **asynchronous event-driven communication** and **publish/subscribe pattern** following to **eventual consistency**.
- Not enough for **complex** and **long E2E business** case that required to visit more that 5 microservices in a distributed environment.
- **Microservices Transactional Boundaries** is identify **minimum smallest unit of atomicity** use cases and design communication between those boundaries.



E-Commerce Microservices Transactional Boundaries

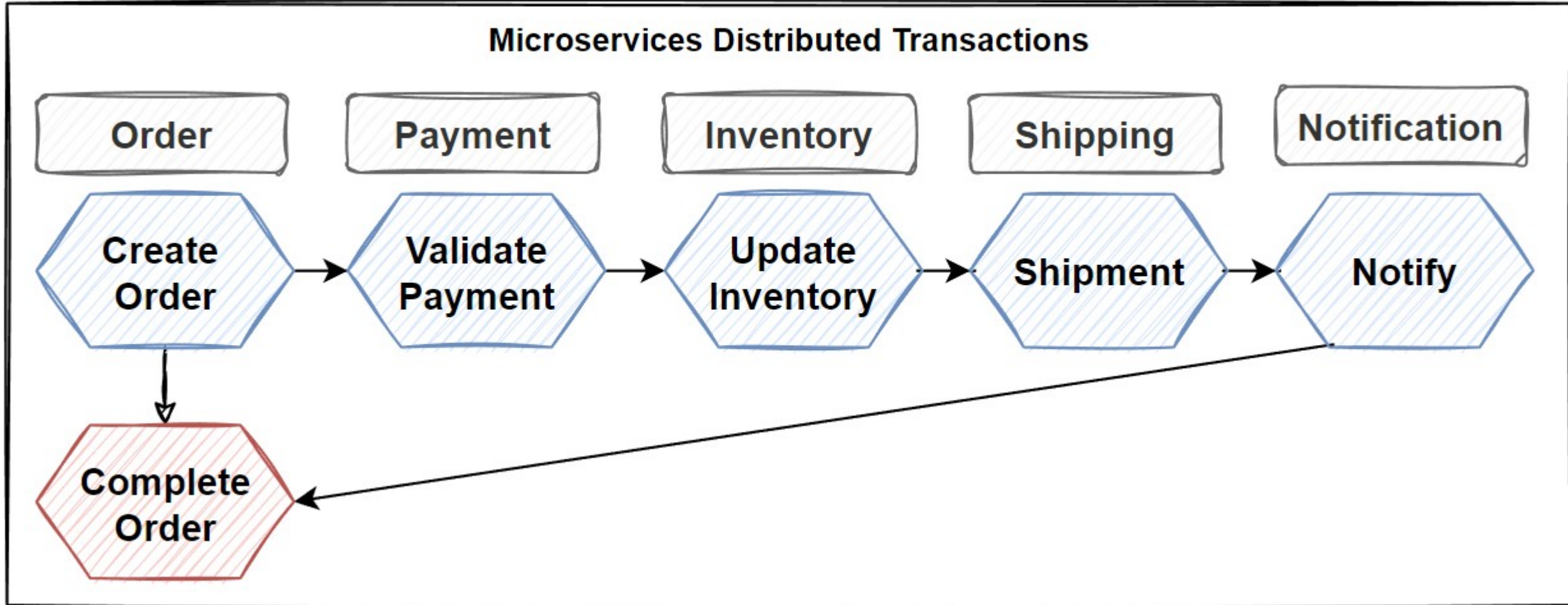
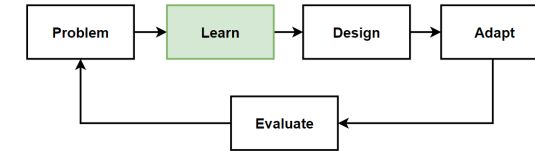


- Customer checkout Shopping Cart and start Ordering process
- Order created into Order microservices and start E2E fulfillment process
- Payment should be validate and done
- Deduct items into Inventory with update Inventory Service
- Shipping to Customer Address manages from Shipment Service
- Notification send to Customer from Notification Service



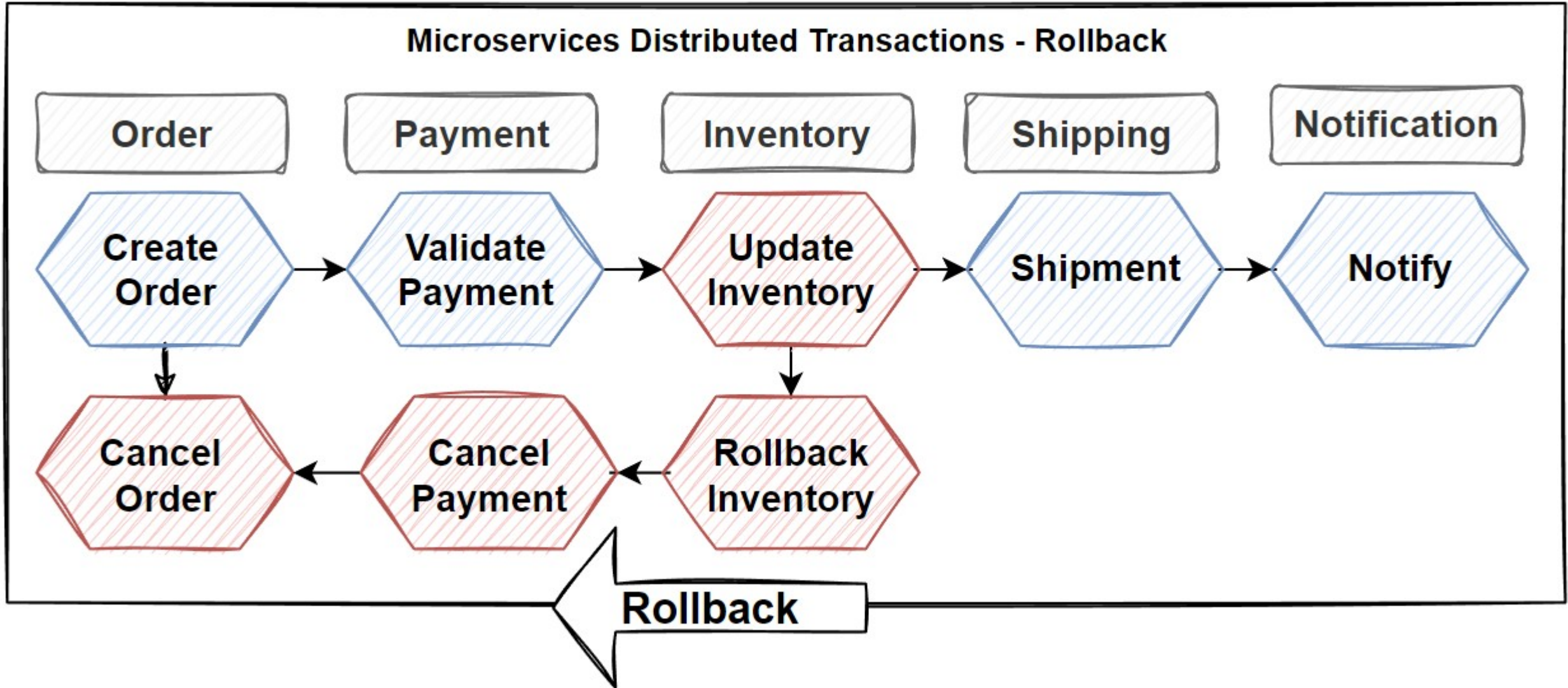
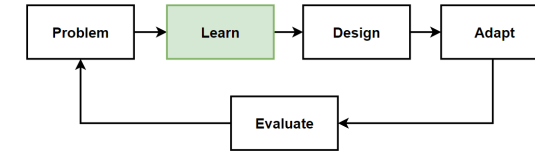
Microservices Transactional Boundaries

Order Fulfillment - Success



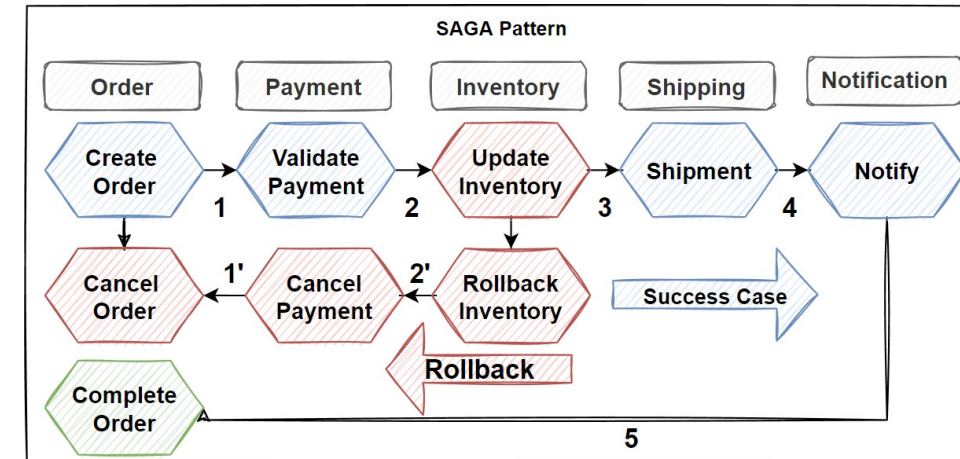
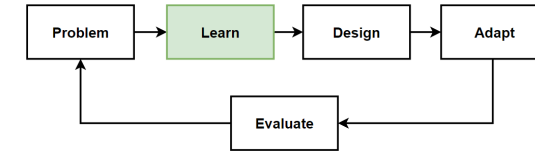
Microservices Transactional Boundaries

Order Fulfillment - Rollback

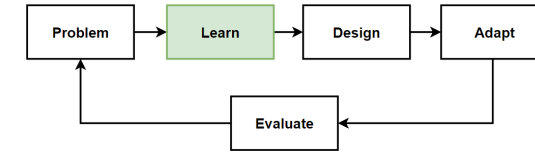


Saga Pattern for Distributed Transactions

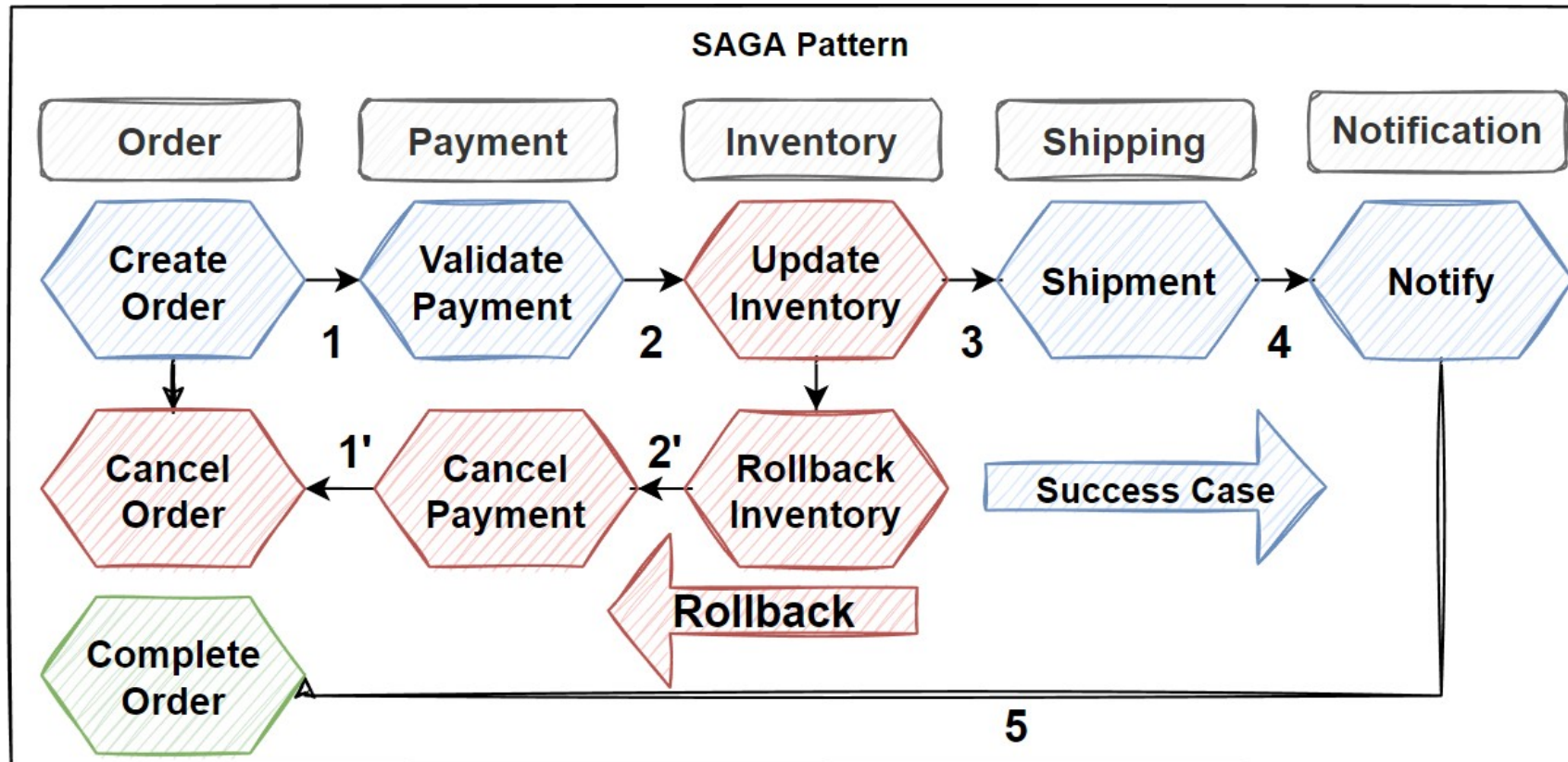
- **Saga design pattern** is provide to **manage data consistency** across microservices in **distributed transaction** cases.
- **Saga** offers to **create a set of transactions** that **update microservices sequentially**, and **publish events** to **trigger the next transaction** for the next microservices.
- If **one of the step is failed**, than saga patterns **trigger to rollback transactions**, do **reverse operations** with **publishing rollback events** to previous microservices.
- **Publish/subscribe pattern** with brokers or **API composition**.
- **SAGA pattern** manage **long-running transactions** that involve multiple microservices which is a **series of local transactions** that work together to achieve **E2E use case**.
- Useful in distributed systems, where **multiple microservices** need to **coordinate their actions**.
- Ensure that the **overall transaction is either completed successfully**, or is **rolled back** to its **initial state**. (compensating transaction)



Saga Pattern for Distributed Transactions - 2

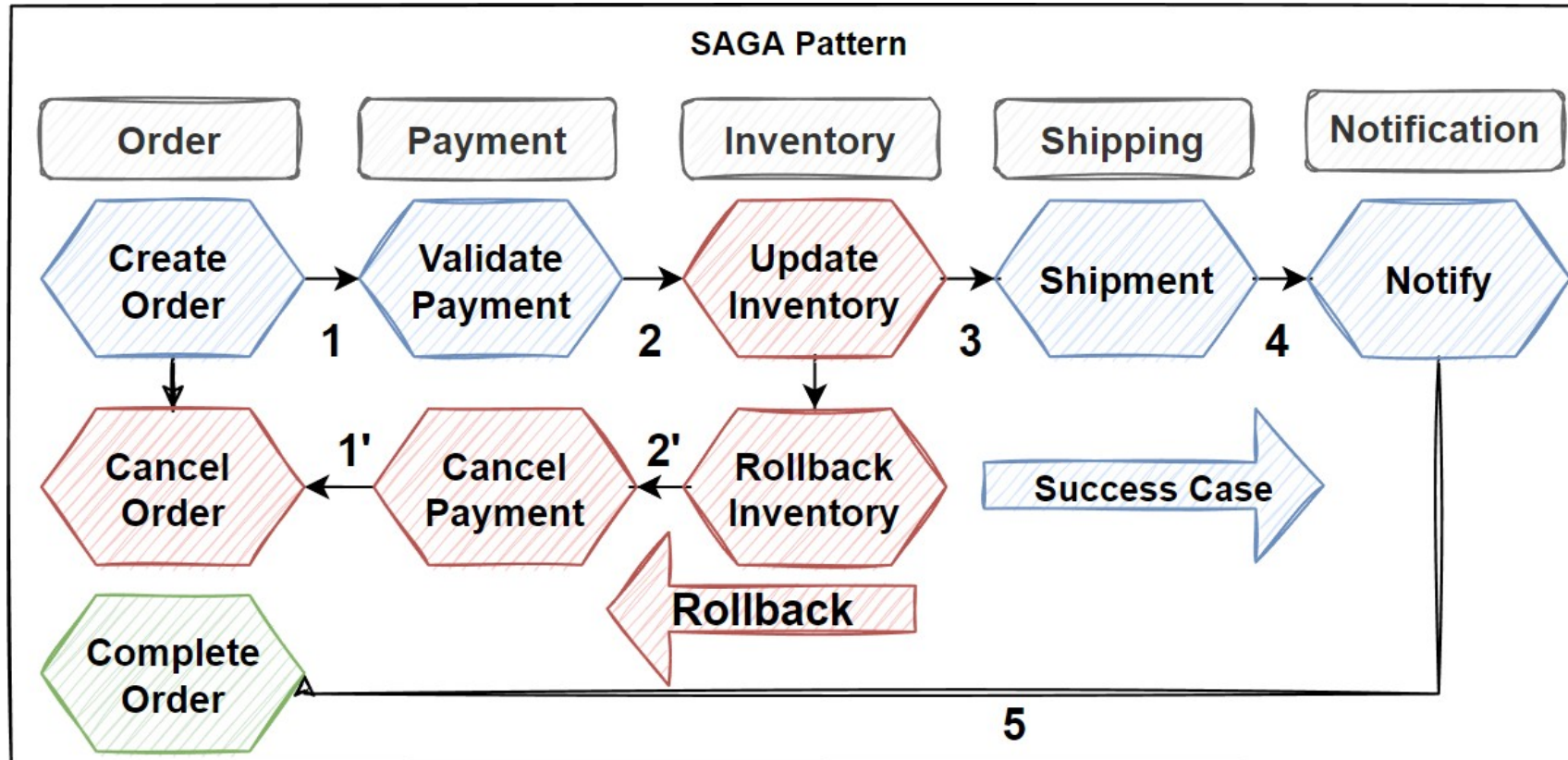
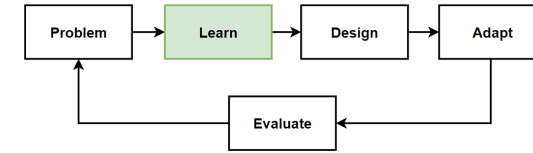


- Saga pattern provides transaction management with **using a sequence of local transactions** of microservices. And **grouping these local transactions** and **sequentially invoking** one by one.
- Each **local transaction updates the database** and **publishes an event** to trigger the **next local transaction**. If **one of the step is failed**, than saga patterns **trigger to rollback transactions**.

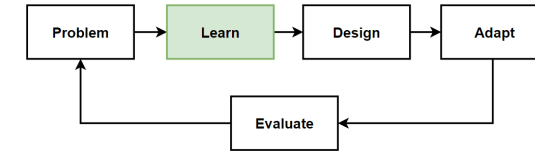


Types of Saga Implementation

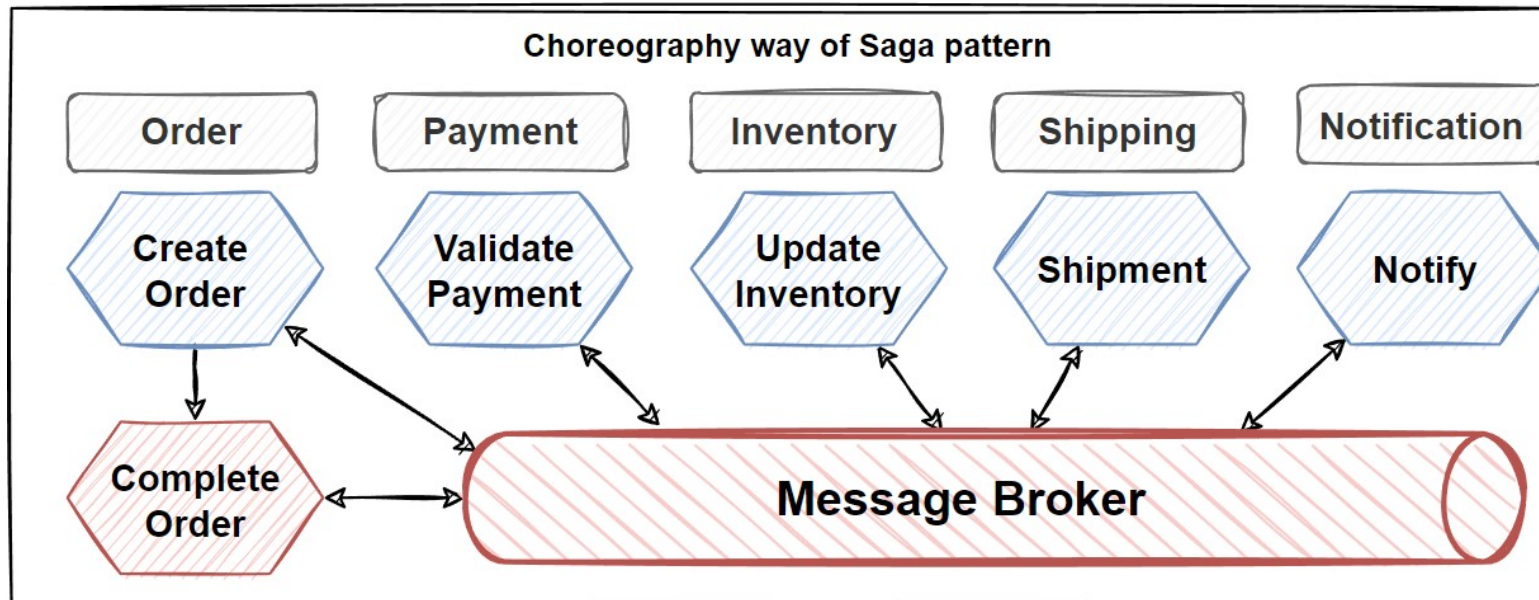
- There are **two type of saga** implementation ways:
- **Choreography-based SAGA Implementation**
- **Orchestration-based SAGA Implementation**



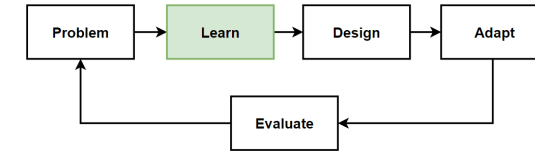
Choreography-based SAGA Implementation



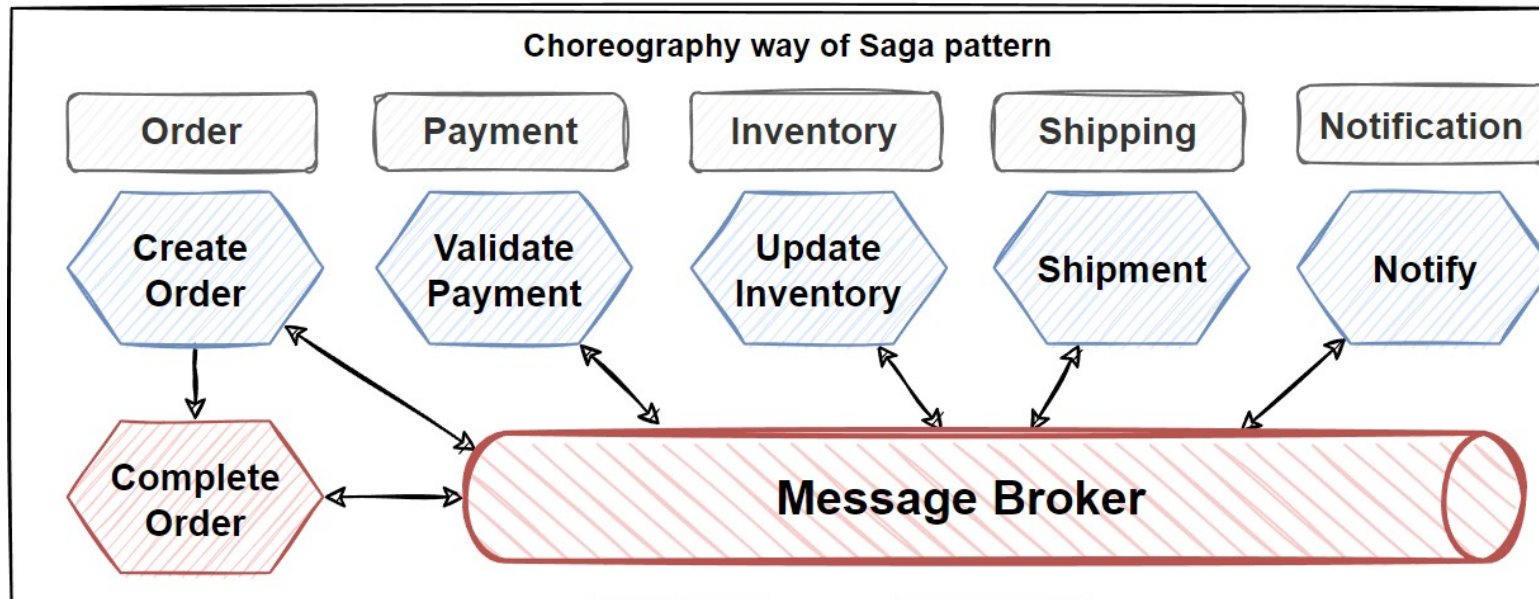
- **Each microservice communicates** with the other microservices by **exchanging events** using a message broker.
- **Event-based approach** allows for a **more decentralized** and **flexible** way to implement the SAGA pattern, as each microservice can **publish to events** to the others **in real-time**.
- **Choreography** provides to **coordinate sagas** with applying **publish-subscribe principles**.
- **Each microservices** run its **own local transaction**, **publishes events** to **message broker** system and **trigger local transactions** in other microservices.



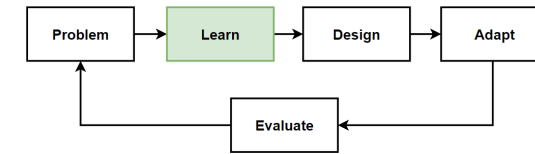
Order Fulfilment with Choreography-based SAGA Implementation



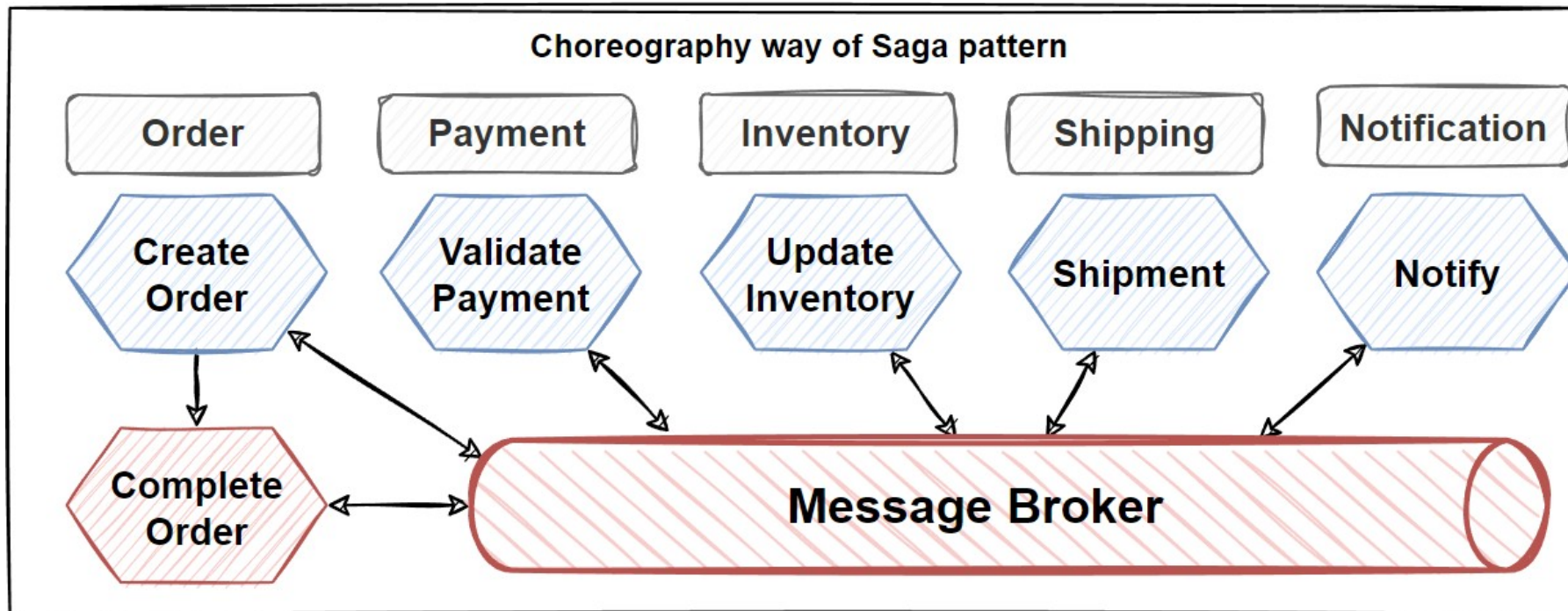
- The customer **places an order** on the e-commerce application and provides their payment information.
- The order microservice **begins a local transaction** and publishes an event to the message broker.
- The inventory microservice **listens for the event published** by the order microservice and, it begins a local transaction and reserves the items in the customer's order.
- The inventory microservice publishes an event to the message broker, the items were reserved successfully.
- The payment microservice **listens for the event** published by the inventory microservice and it charges the customer's payment method and commits its own local transaction.



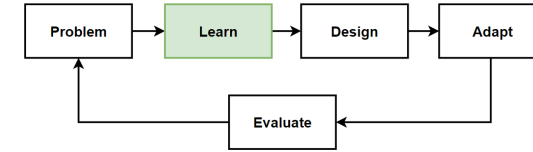
Order Fullfilment Rollback with Choreography-based SAGA Implementation



- SAGA pattern provides a way to **roll back** the **changes** made by each microservice.
- If the Inventory microservice encountered an error while reserving the items in the customer's order, it could **publish a failure event** to the message broker.
- Order fulfillment microservice would then **execute a compensating transaction** to undo the charges to the customer's payment method and cancel the order.



Benefits of Choreography-based SAGA



- **Decentralized and flexible**

By using an event-based approach, each microservice can react to events published with the others to coordinate their actions. This allows for a more decentralized and flexible approach to implementing the SAGA pattern.

- Decouple direct dependency of microservices when managing transactions.

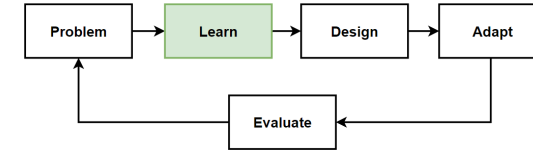
- **Avoid Single Point of Failure**

Since there is no orchestrator, responsibilities are distributed across the saga participants.

- **Simple workflows**

This way is good for simple workflows, if they don't require too much microservices transaction steps.

Drawbacks of Choreography-based SAGA



- **More complex to manage**

Each microservice communicates via events with the others, the choreography-based approach can be more complex to manage and may require more coordination among the microservices.

- SAGA workflow become confusing when adding new steps into flow.

- **Cyclic Event Consume Risk**

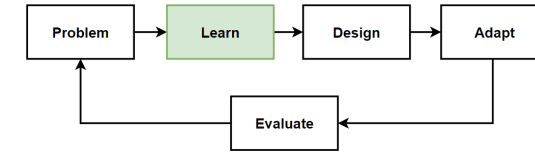
There's a Cyclic Event Consume Risk dependency between saga participants because they have to consume each other's commands.

- **Result**

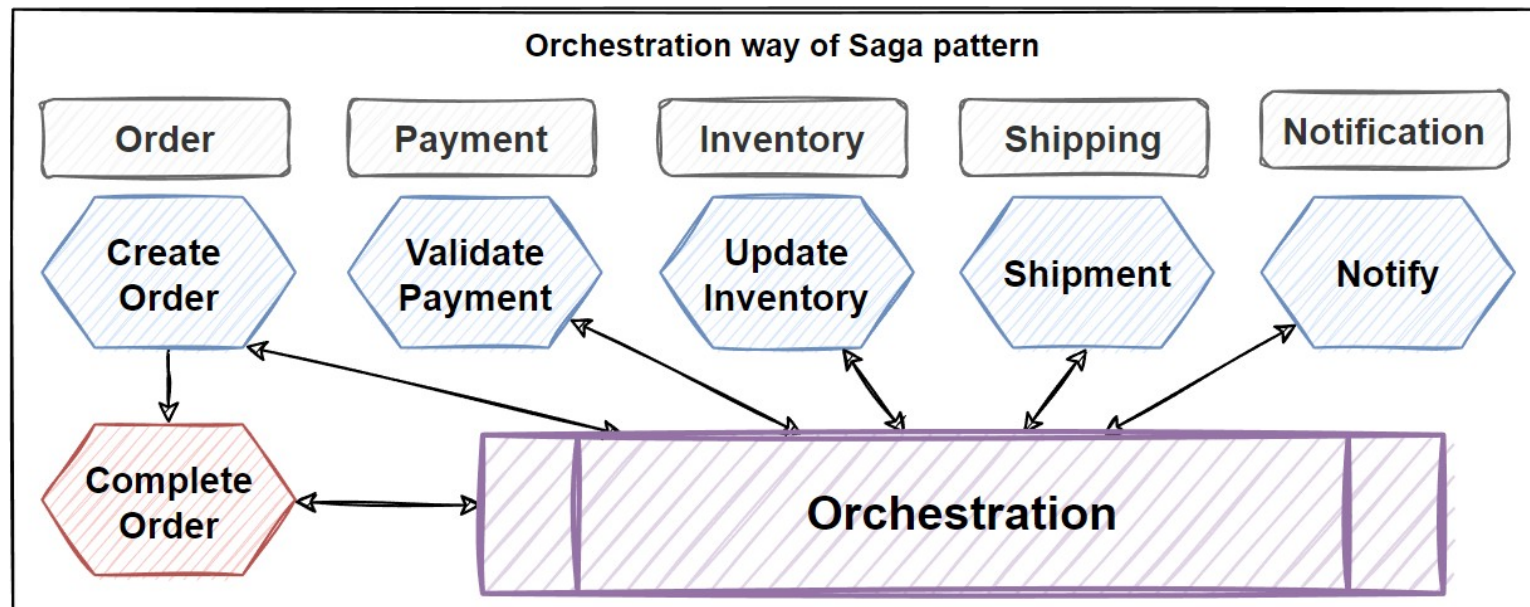
Implementing the SAGA pattern will depend on the specific needs and constraints of the distributed system. It may be a good fit for some systems, but may not be suitable for others.

- Choreography-based implementation of SAGA is good for simple workflows if they don't require too much microservices transaction steps.

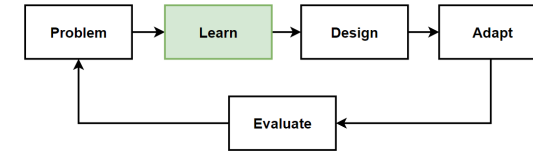
Orchestration-based SAGA Implementation



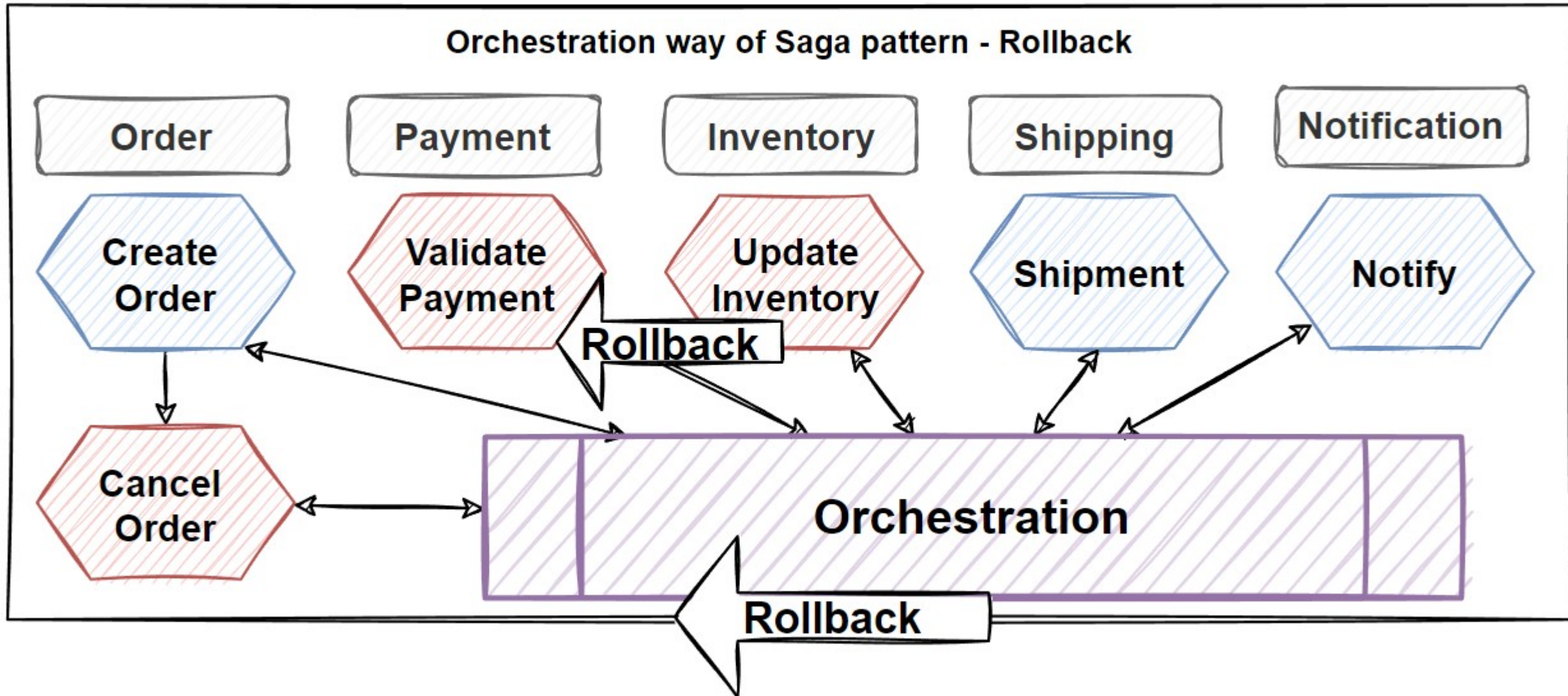
- **Orchestration-based SAGA pattern** involves using a **central orchestrator** service to **coordinate** and **manage** the **individual sagas** or microservices that make up a transaction.
- The orchestrator is responsible for **initiating the transaction** and **ensuring** that each **saga performs** its step in the **correct order**.
- **If any of the sagas fail** to complete their step, the **orchestrator** can use the **compensating transactions** to **roll back the changes** and **restore** the system to its **original state**.
- **Orchestration** provides to **coordinate sagas** with a **centralized controller** microservice that orchestrate the saga workflow and invoke to **execute local** microservices **transactions** in **sequentially**.



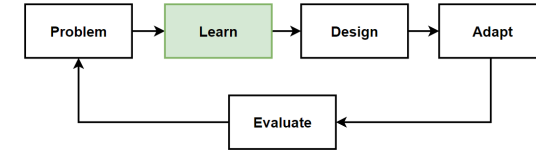
Orchestration-based SAGA Implementation - Rollback



Orchestration way of Saga pattern - Rollback



Benefits and Drawback of Orchestration-based SAGA Implementation



Benefits

- Provides a clear and centralized point of control for managing transactions.
- Make it easier to understand and debug the system, and to add new transactions or modify existing ones.

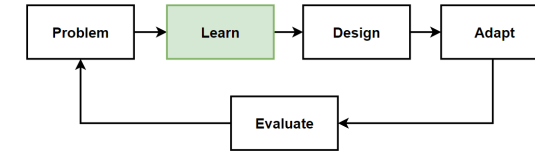
Drawbacks

- The orchestrator can also become a single point of failure, and if it goes down, the entire system may be unable to complete transactions.
- The orchestrator can become a bottleneck if the system is heavily loaded, as all transactions must go through it.

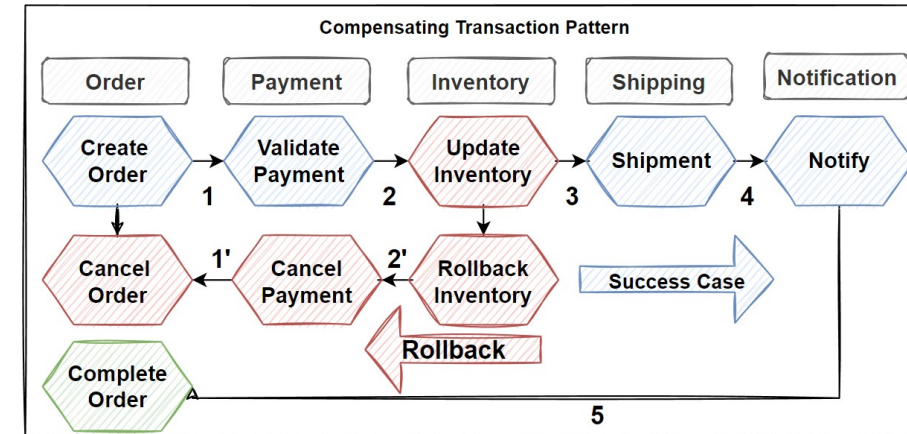
Result

- It can be a useful approach for managing transactions in a distributed system, but it is important to carefully consider the trade-offs and potential drawbacks.
- Orchestration way is good for complex workflows which includes lots of steps.
- But this makes single point-of-failure with centralized controller microservices and need implementation of complex steps.

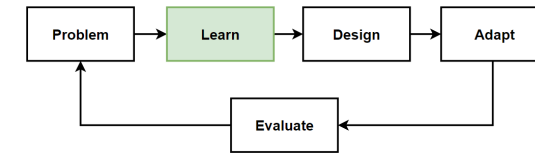
Compensating Transaction Pattern



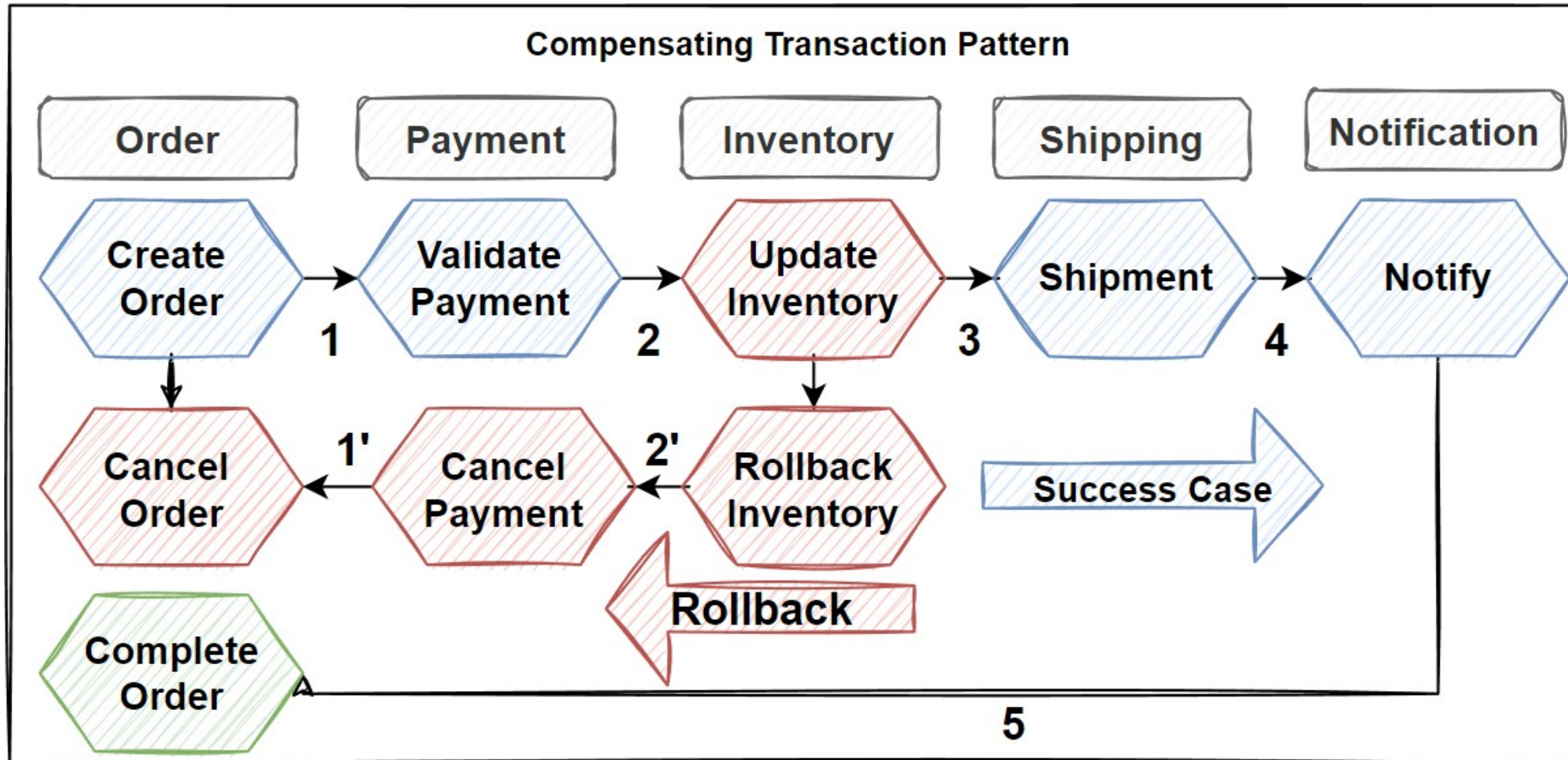
- **Compensating Transaction pattern** is a rollback process of **SAGA Pattern**.
- Compensating Transaction pattern provides to **reverse the steps** for a **previously executed transaction**.
- In microservice architectures where **multiple services** may be involved in a **distributed transaction**, if **any of the services fail** to complete their part of the transaction, the effects of the **entire transaction need** to be **undone**.
- The steps in a **compensating transaction** should **undo the effects** of the steps in the original operation.
- Compensating transaction is also an **eventually consistent** operation and it could fail.
- The system should be able to **resume the compensating transaction** at the point of failure and continue.
- The steps in a compensating transaction should be defined as **idempotent commands**.



Ecommerce Order Fulfilment Compensating Transaction Pattern



- Customer places an order on the e-commerce application

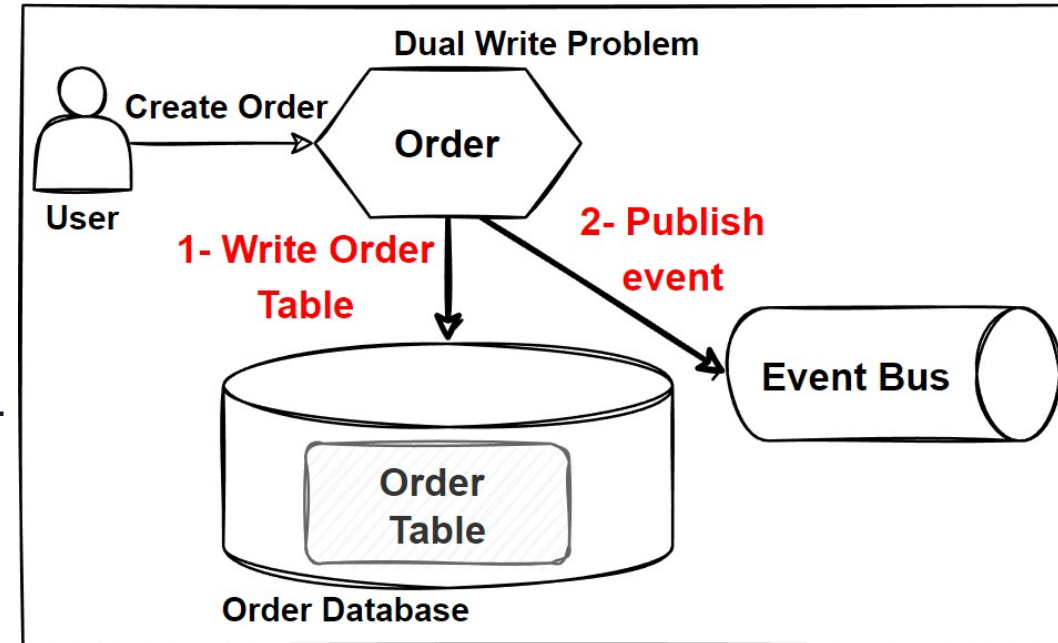
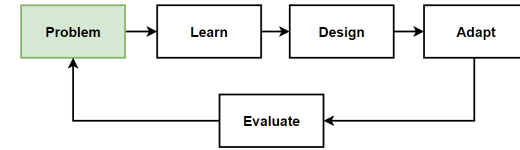


Problem: Dual Write Problem

- When application **needs to change data** in **two different systems**, i.e. a database and a message queue, if one of the writes fails, it can result in inconsistent data.
- Happens when you use a **local transaction** with **each of the external systems** operations.
- I.e. app needs to **persist data** in the **database** and **send a message** to **Kafka** for notifying other systems.
- **If one of these two operations fails**, the **data** will be **inconsistent** data and these two systems becomes inconsistent.

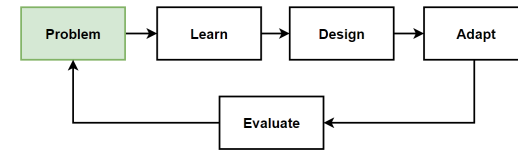
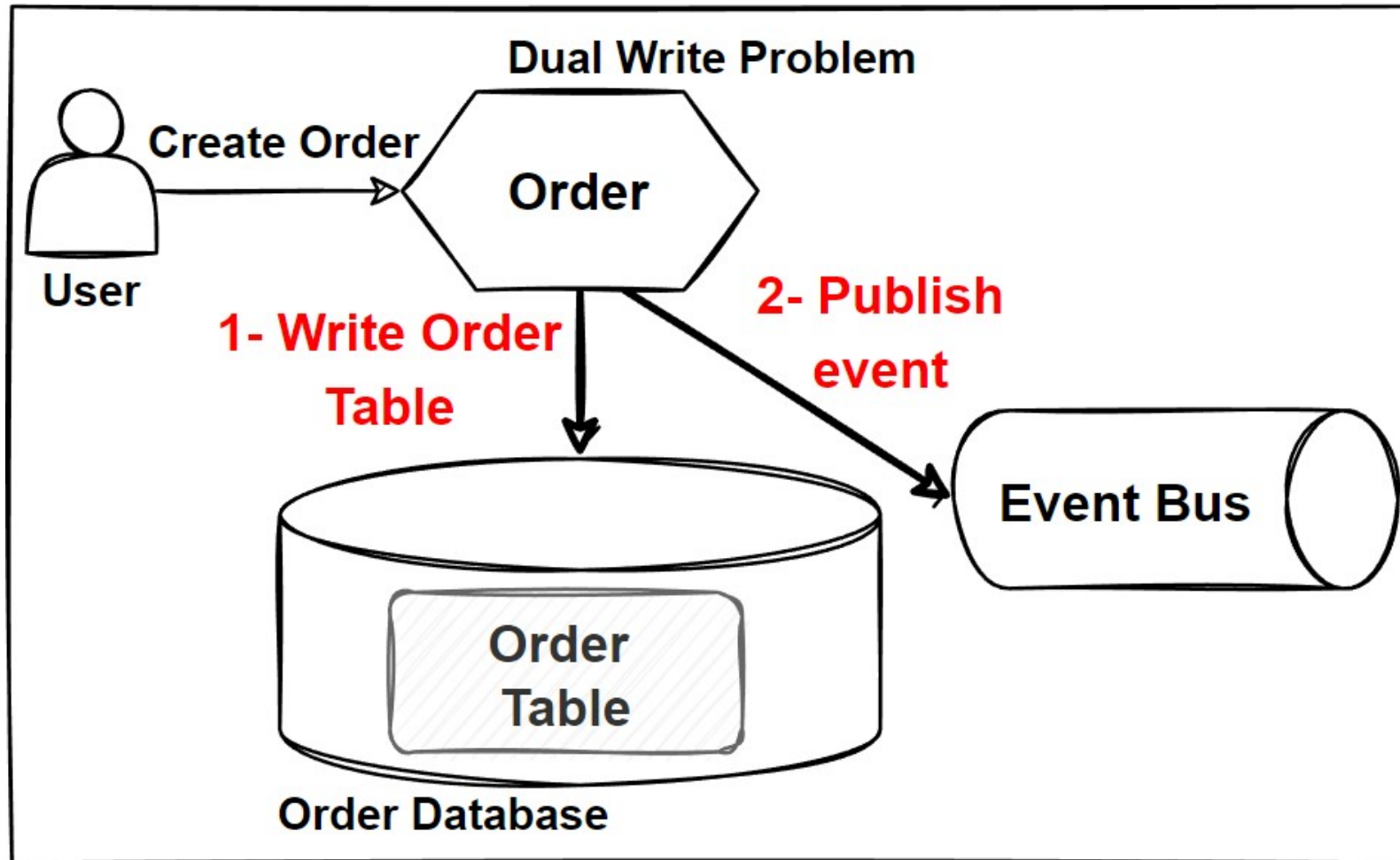
Problems

- Data loss or corruption.
- Difficult to resolve without proper error handling and recovery mechanisms.
- Dual writes can be hard to detect and fix.



Example of Dual-write problem: Create Order

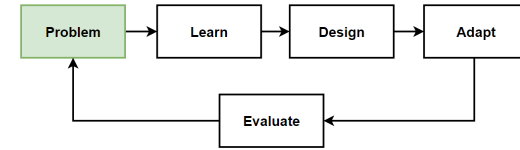
- 1 Change data in Order database with creating new record
- 2 Send an order_created event to the EventBus like Apache Kafka.



Run Transactions

- 1, 2 or
- 2, 1
- Both becomes dual writes

How to avoid dual write problems in microservices ?



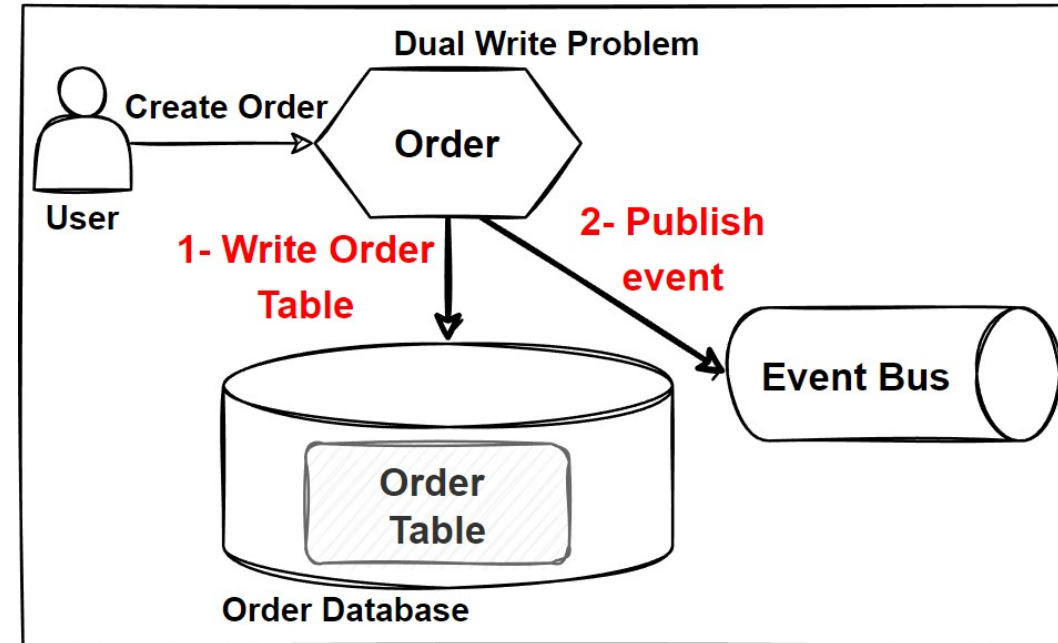
- **Monolith** applications use the **2 phase commit** protocol.
- It **splits the commit** process of the transaction into 2 steps and ensures the **ACID principles** for all systems.
- **Can't use 2-phase commit** transactions when building **microservices**.
- These transactions **require locks** and **don't scale well**.
- Need all systems to be **up and running** at the same time.

Solutions

- Transactional Outbox Pattern
- CDC - Change Data Capture

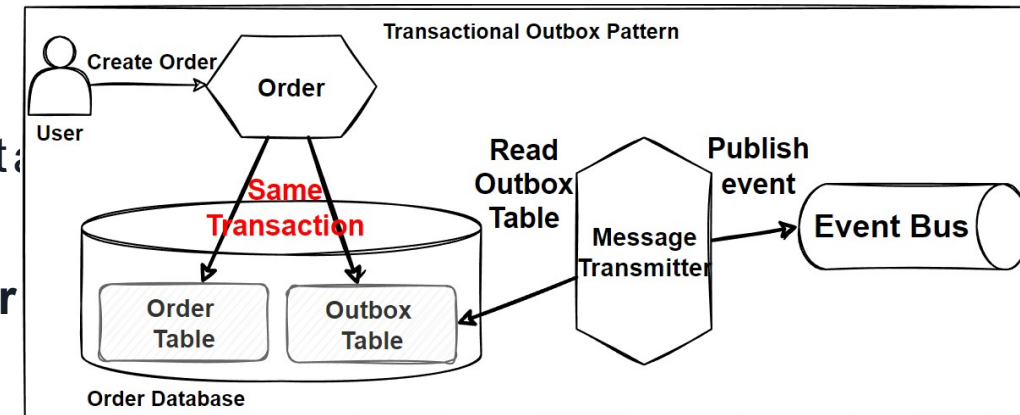
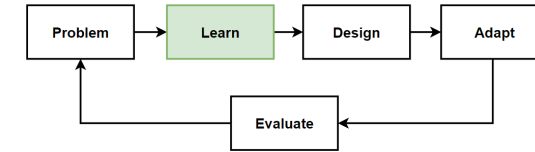
Best Practice

- Use Red Hat **Apache Kafka** and **CDC** using **Debezium** in event-driven applications.
- Use New databases like **CockroachDB** which has built-in **Change Data Capture** feature.

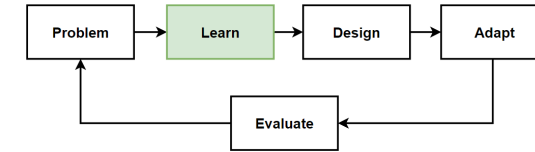


Transactional Outbox Pattern

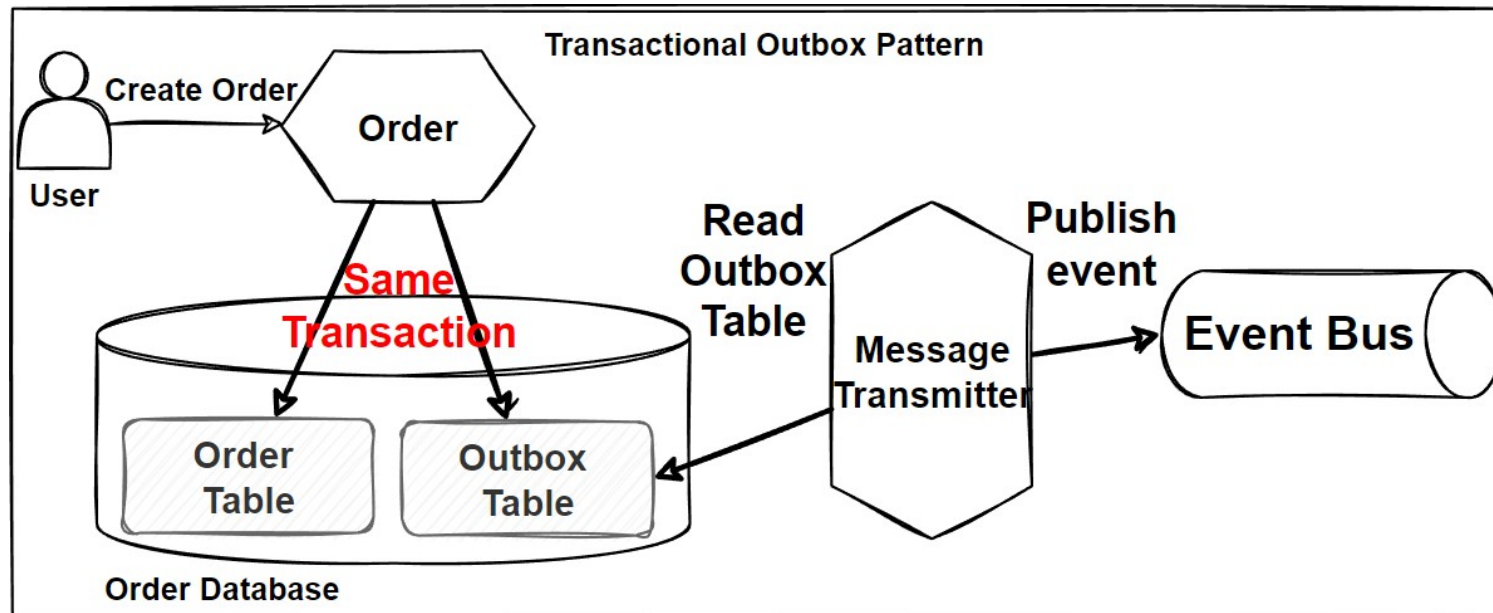
- The idea is to have an “**Outbox**” **table** in the microservice’s database. It provides to publish events reliably.
- **Dual write problem** happens when application needs to change data in two different systems.
- Instead of sending the data to two separate locations, **send a single transaction** that will **store two separate copies** of the data on the database.
- **One copy** is **stored** in the relevant **database table**, and the **other copy** is **stored** in an **outbox table** that will publish to event bus.
- When API **publishes event messages**, it doesn’t directly send them, Instead, the **messages** are **persisted** in a **database table**.
- After that, **a job publish events** to message broker system in predefined time intervals.
- **Events** are **not written directly** to a event bus, it is written to a table in the “**Outbox**” role of the service.



Transactional Outbox Pattern - 2

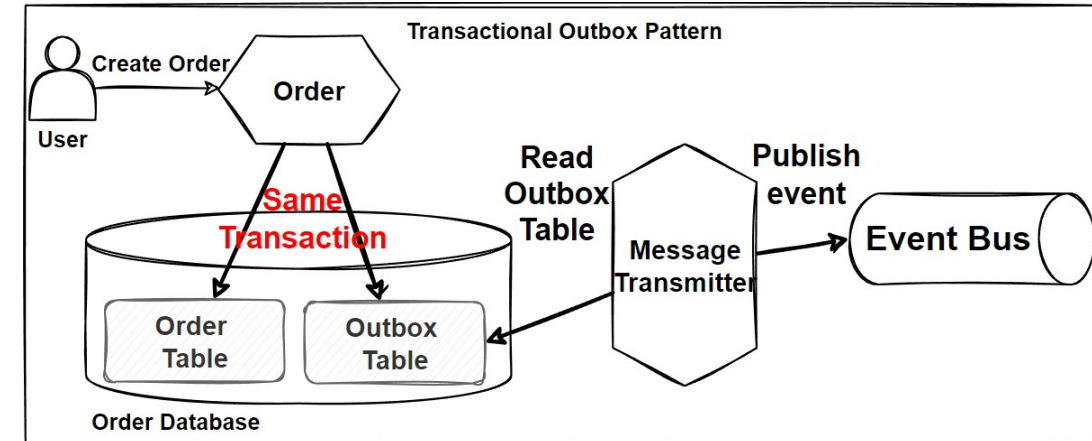
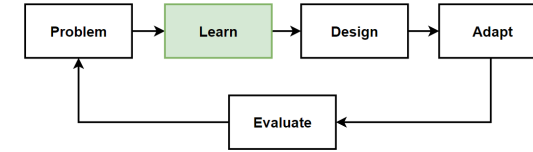


- Transaction **performed before the event** and the **event written to the outbox table** are part of the **same transaction**.
- When a **new order is added** to the system, the process of adding the order and writing the **Order_Created event** to the **Outbox table** is done **in the same transaction** to ensure the event is saved to the database.
- If one of the **process is fail**, this will **rollback the whole operations** with following ACID principles.
- The second step is to **receive these events** written to the **Outbox table** by an **independent service** and **write** them to the **Event bus**. Another service **listen** and **polls the Outbox table** records and publish events.



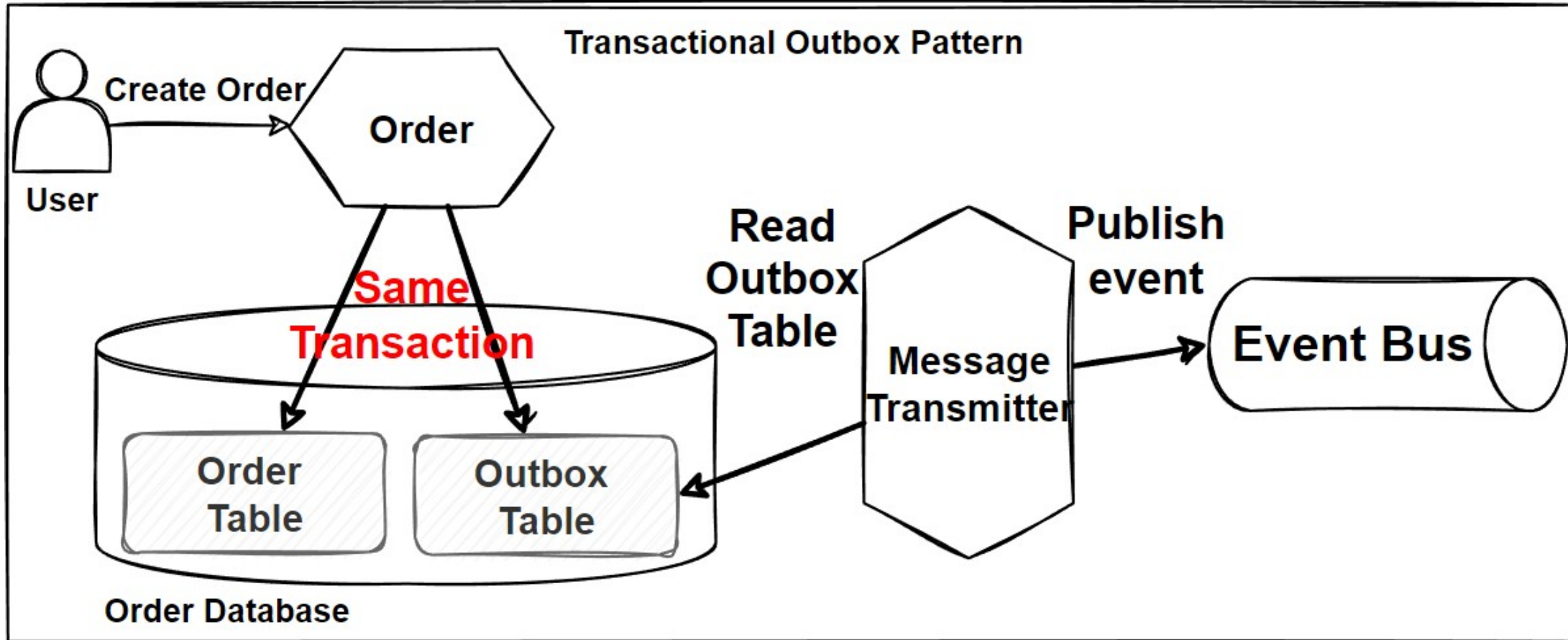
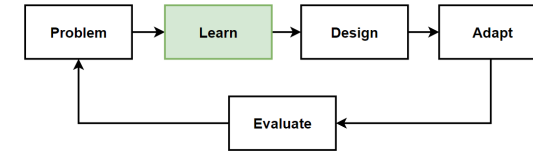
Transactional Outbox Pattern in Microservices

- **Microservice** provides an **outbox table** within its database. Outbox table will **include** all the **events**.
- There will be a **CDC (change data capture) plugin** that **reads** the **commit log** of the **outbox table** and **publish the events** to the relevant queue.
- It provides that **messages** are **reliably delivered** from a **microservice** to **another microservice** even if the transaction that triggered the message fails.
- It involves **storing** the **message** in a local "**Outbox**" table within the microservice, that message sent to the consumer after the transaction is committed.
- **Outbox pattern** can be used to ensure that **messages** are **delivered consistently**, even if the microservice that sent the message is **unavailable** or **experiencing errors**.
- Useful for **communicating important information** or updates between services.



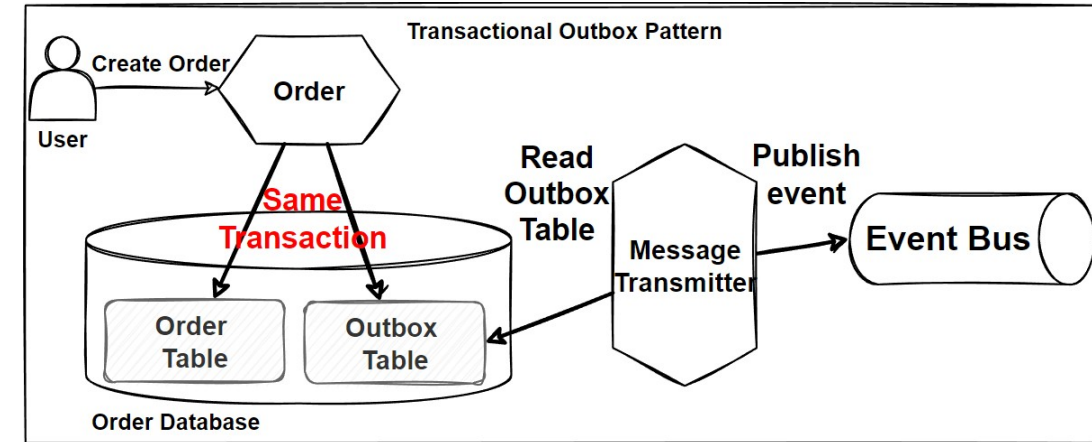
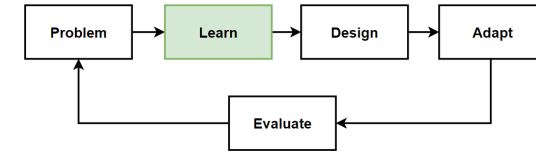
Customer Places an Order Use Case

Transactional Outbox pattern in Microservices



Why Use Outbox Pattern ?

- When **working with critical data** that **need to consistent** and **need to accurate** to catch all requests.
- When the **database update** and **sending of the message** should be **atomic** to make sure **data consistency**.
- For example the **order sale transactions**, because they are about **financial business**. Thus, the calculations **must be correct 100%**.
- To access this **accuracy**, must be sure that our system is not **losing any event messages**.
- **The Outbox Pattern** should be applied this kind of cases.



Problem: Listen and Polling Outbox Table

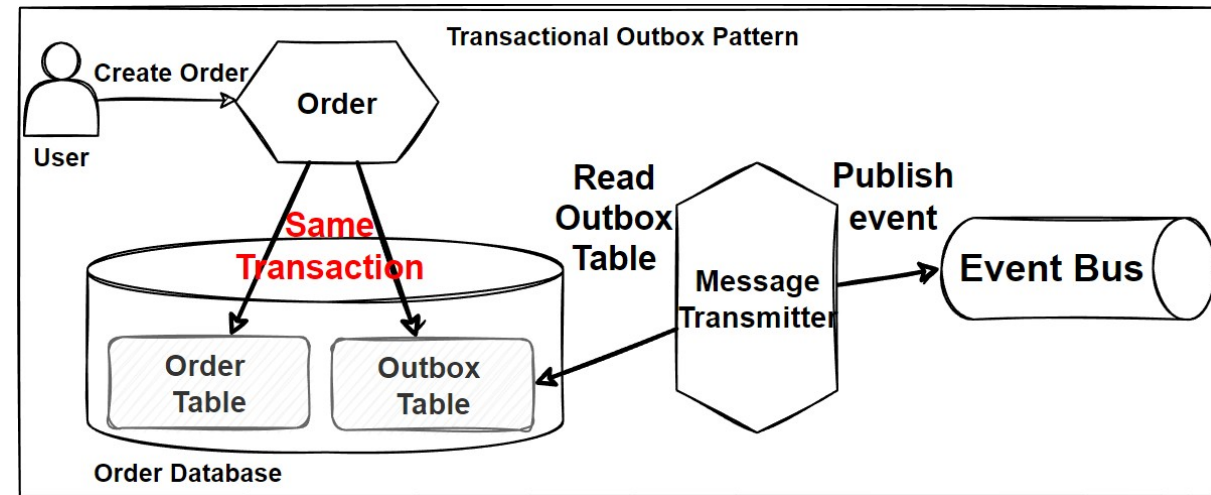
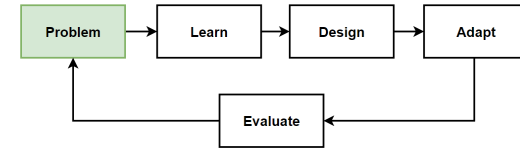
- **Creating new order record** into order table and **creating new order_created event** into outbox table **are in the same transactions**.
- Another service **listen** and **polls** the **outbox table records** and publish events into separate processes.
- The current architecture **required additional microservices** that listen and polls records from the outbox table.

Problems

- Reduces the performance and error-prone operations.
- Time is wasted during Polling, wasted resources.
- Consuming system resource unnecessary pull requests.
- Come with a latency overhead, reach system limitations.
- Can cause additional Dual-write problem.

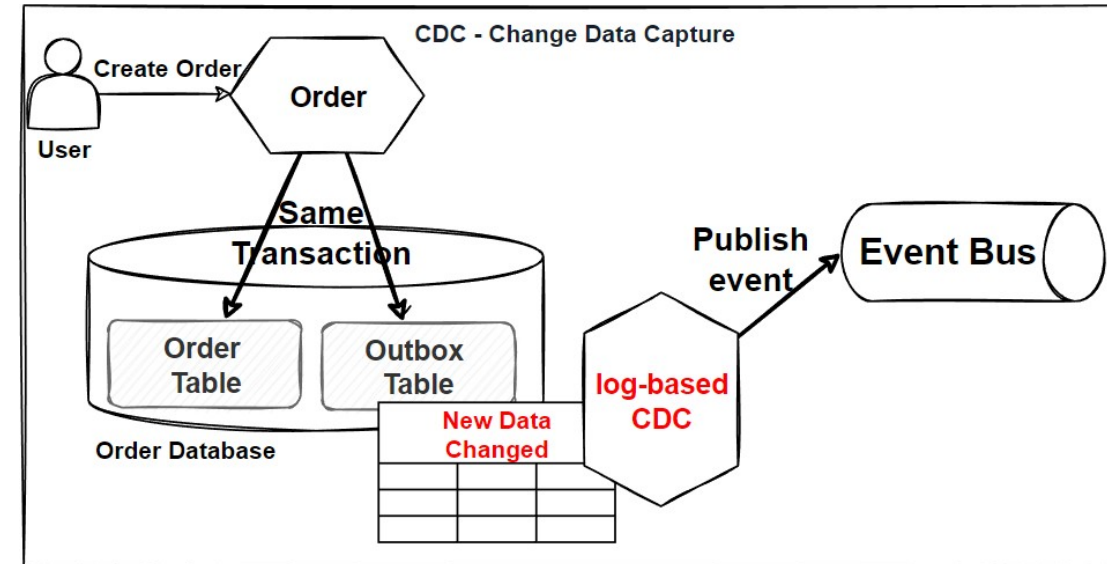
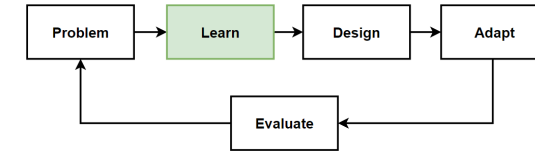
How solve these problems when applied outbox pattern ?

- CDC - Change Data Capture with Outbox Pattern.

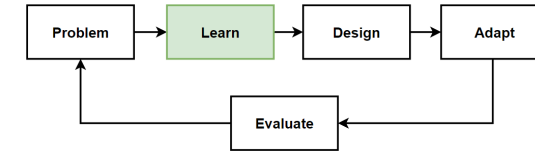


CDC - Change Data Capture

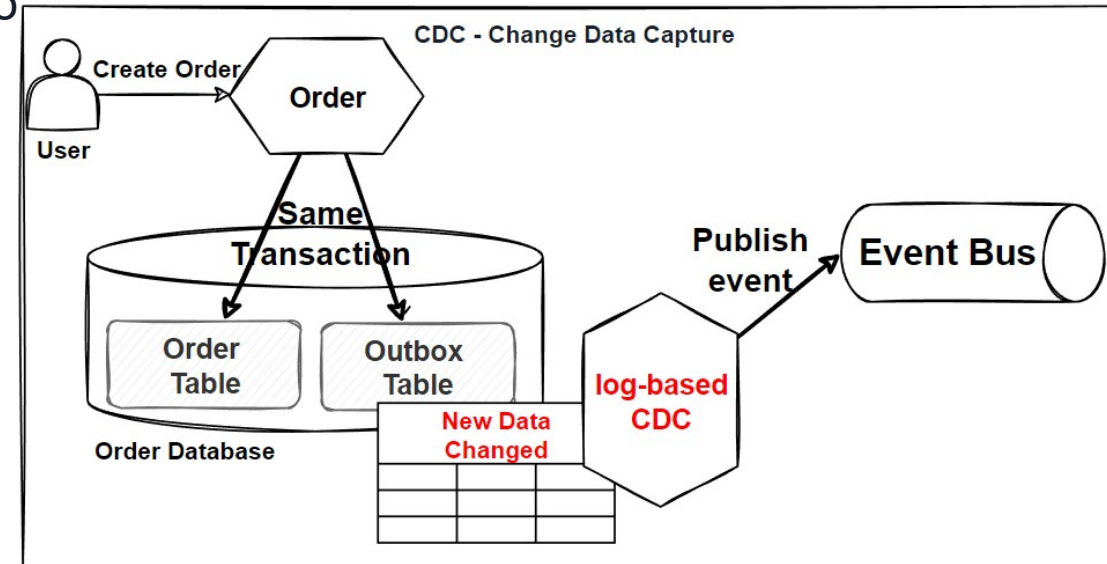
- **Change Data Capture (CDC)** is a technology that captures insert, update, and delete activity on a database.
- **CDC** typically works by **continuously monitoring** the **transaction log** of a **database** for **changes**, and then **extracting** and **propagating** those **changes** to the target system.
- This allows the target system to **stay up-to-date** with the source system **in near real-time**, instead of relying on batch-based data synchronization processes.
- **CDC** can be used in **replicating data between databases**, **synchronizing data** between systems in a microservices architecture, and **enabling real-time data analytics**.
- **CDC** is a way to **track changes** that happen to data in a database that **captures insert, update, and delete activity** and makes this information available to other systems.
- This allows those systems to **stay up-to-date** with the data in the **database in real-time**.



CDC - Change Data Capture with Outbox Pattern

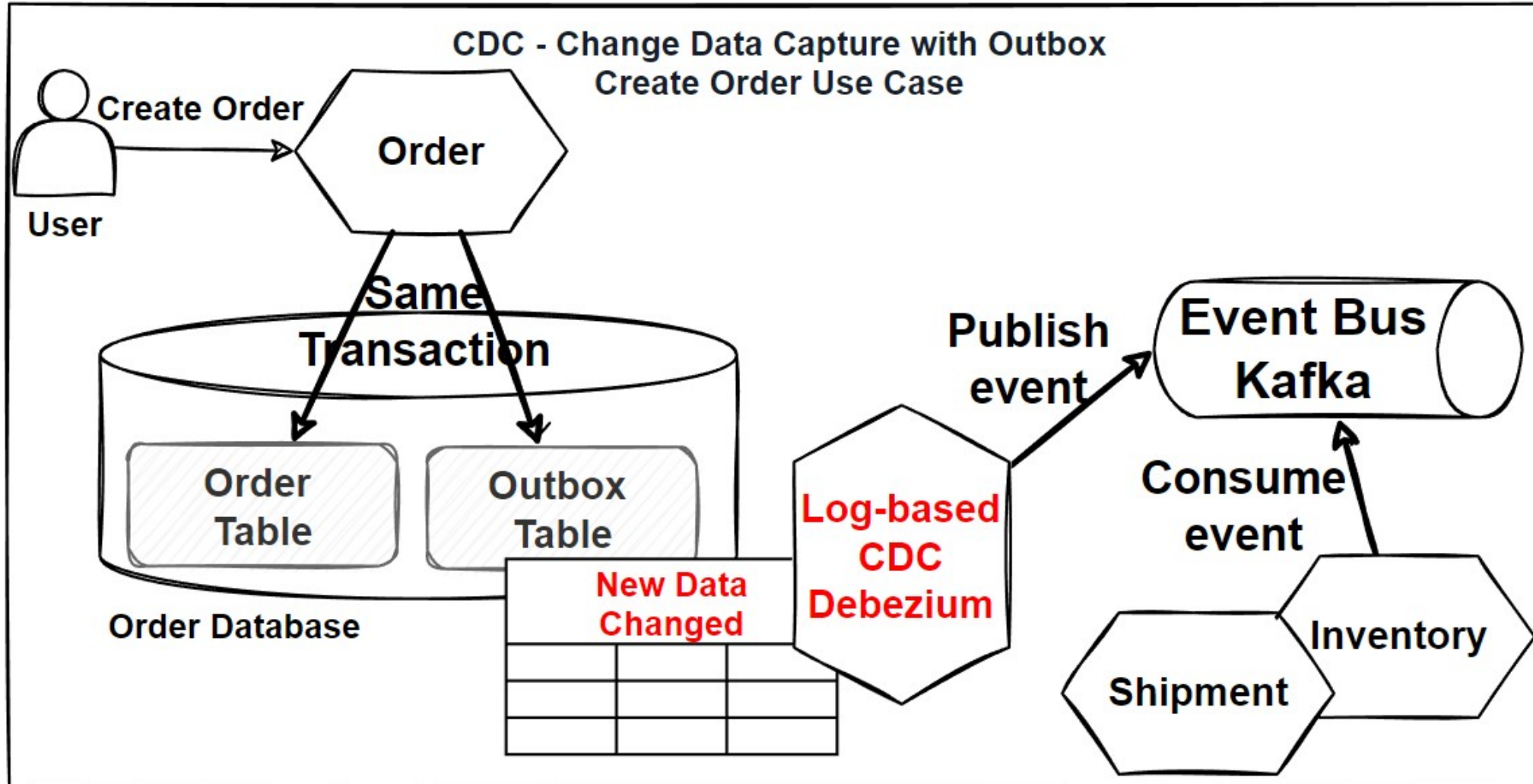
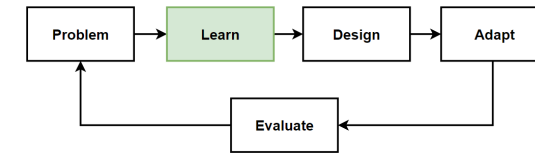


- **Outbox pattern** is ensuring **data changes** made by a microservice are **eventually propagated** to other microservices.
- Whenever a microservice **updates data** in its database, it also writes a **record to the outbox table** with the details of the change.
- **CDC** can then be used to **monitor the outbox table** for new records, **extract the data changes** that **propagated** to the **target microservices** to be **kept up-to-date** with the data.
- Using **CDC** with the **Outbox pattern** allows microservices to **decouple** their **data updates** from the **process** of **propagating** those **updates** to other microservices.
- This can make it **easier to scale** and **maintain** a microservices architecture.
- Each microservice can **focus on its own data updates** and let **CDC handle the synchronization** of data between services.



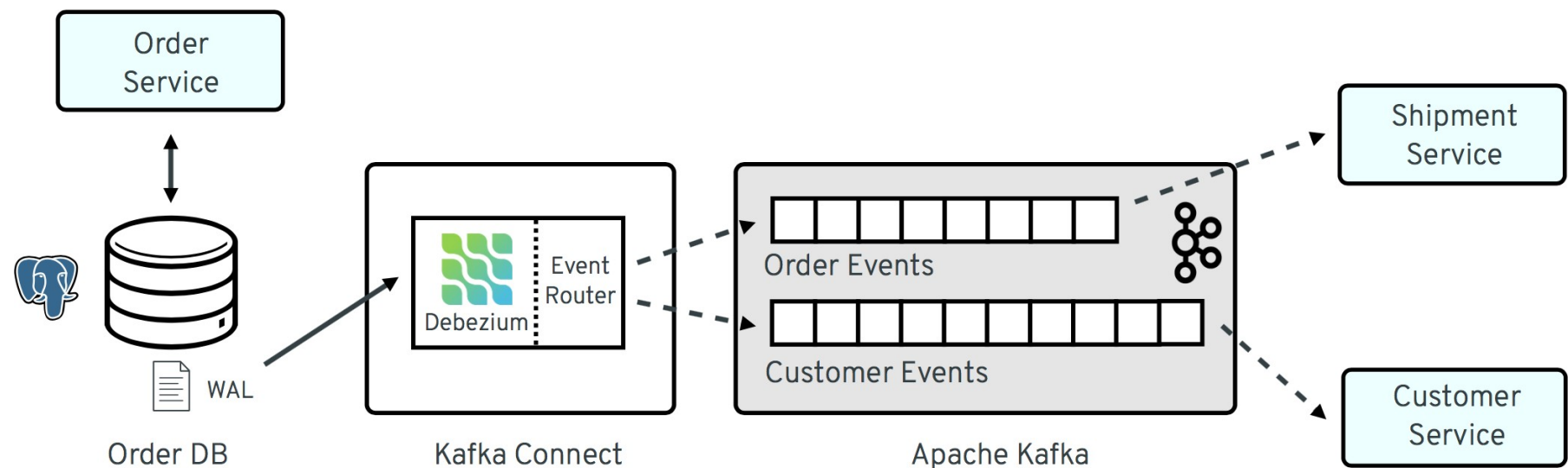
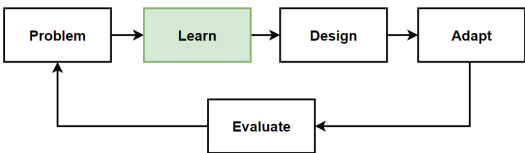
CDC - Change Data Capture with Outbox Pattern

E-commerce Create Order Use Case



CDC - Change Data Capture with Outbox Pattern

E-commerce Create Order Use Case



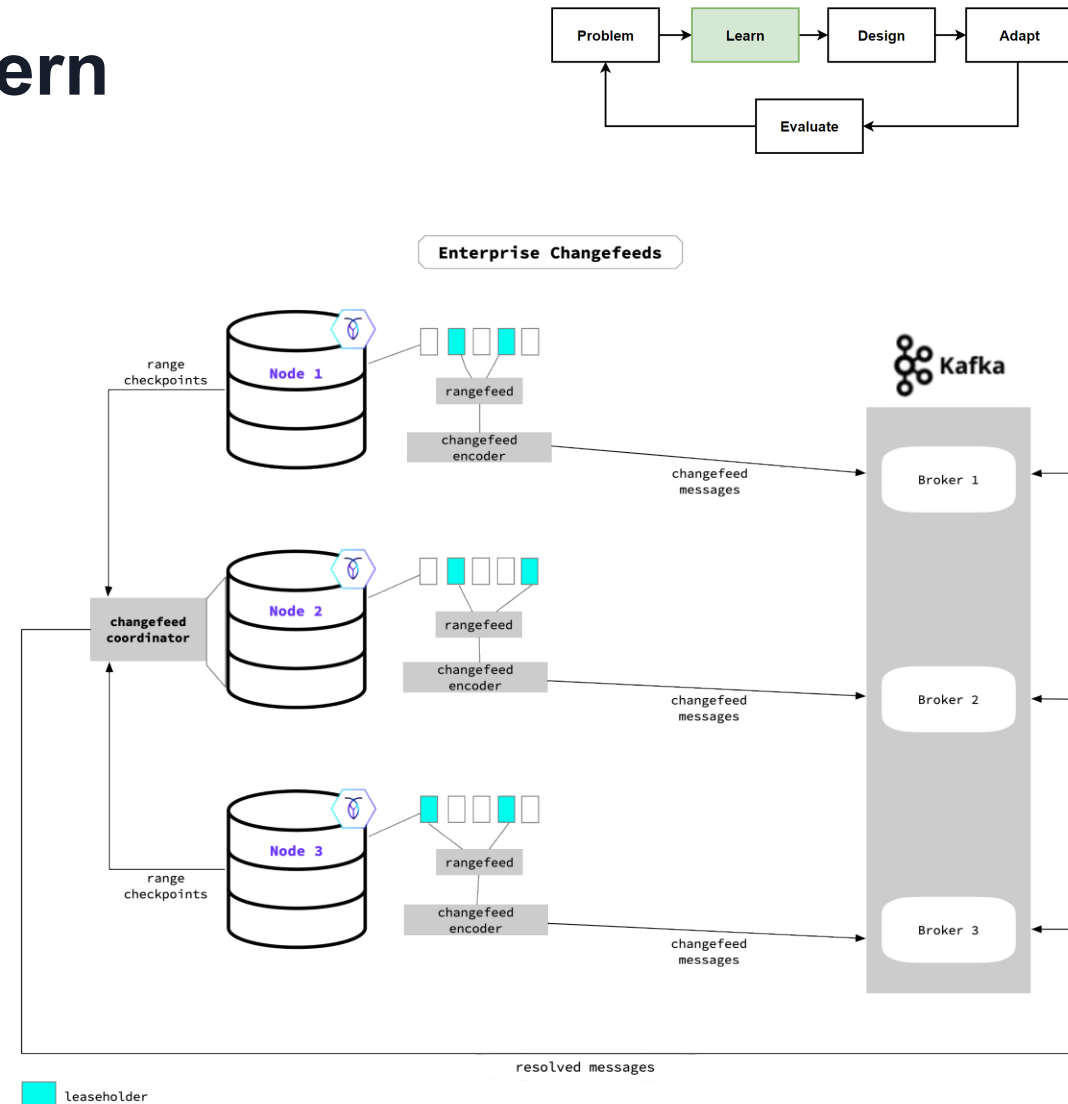
Id	AggregateType	AggregateId	Type	Payload
ec6e	Order	123	OrderCreated	{ "id" : 123, ... }
8af8	Order	456	OrderDetailCanceled	{ "id" : 456, ... }
890b	Customer	789	InvoiceCreated	{ "id" : 789, ... }

Outbox Table

<https://debezium.io/blog/2019/02/19/reliable-microservices-data-exchange-with-the-outbox-pattern/>

CockroachDB for CDC and Outbox Pattern

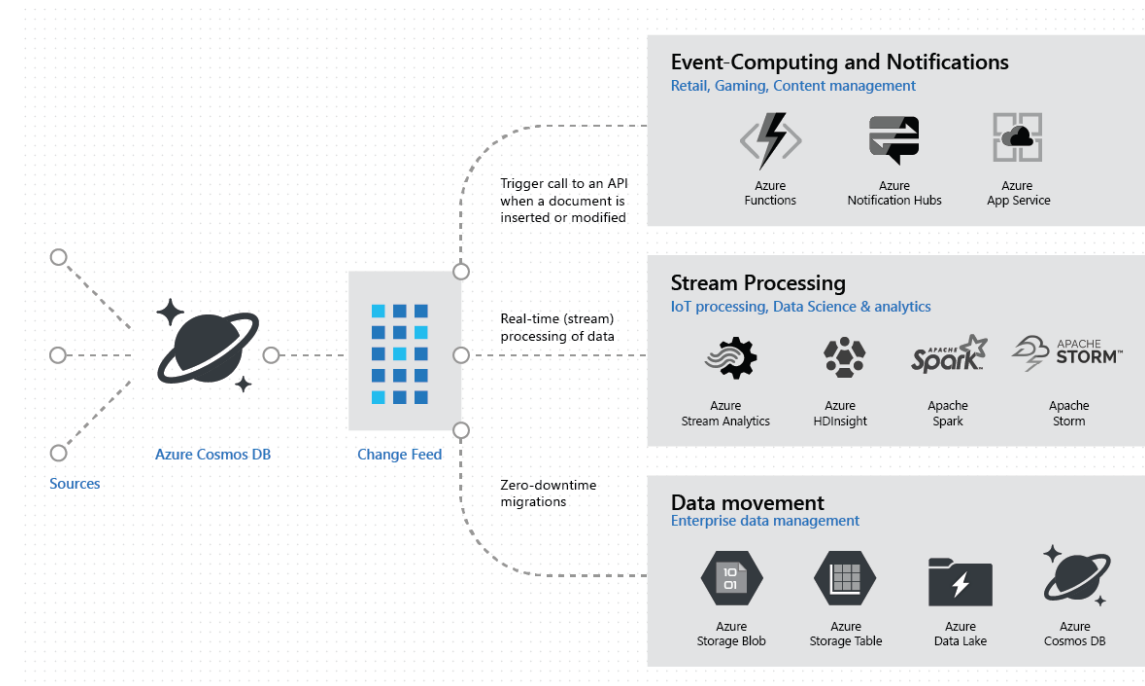
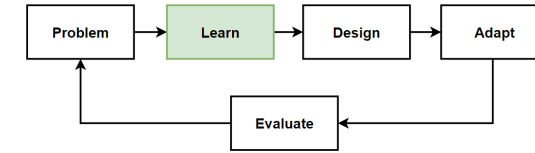
- **CockroachDB** is a **distributed database** management system that is designed to be **scalable**, **resilient**, and easy to use.
- **Cockroach cluster**, which is a **group of database nodes** that work together to form a single, highly available database.
- Ability to **scale horizontally** by **adding more nodes** to the **cluster** as the **workload increases**.
- **CockroachDB** also has strong support for **data consistency** and **durability**, with features such as **multi-active availability** and **distributed transactions**.
- **CockroachDB** is written in the **Go programming** language.
- **CockroachDB** has **built-in Change Data Capture** feature, that you can build the **Transactional Outbox Pattern** with **CDC** into your own application.



<https://www.cockroachlabs.com/docs/stable/change-data-capture-overview.html>

Azure Cosmos DB for CDC and Outbox Pattern

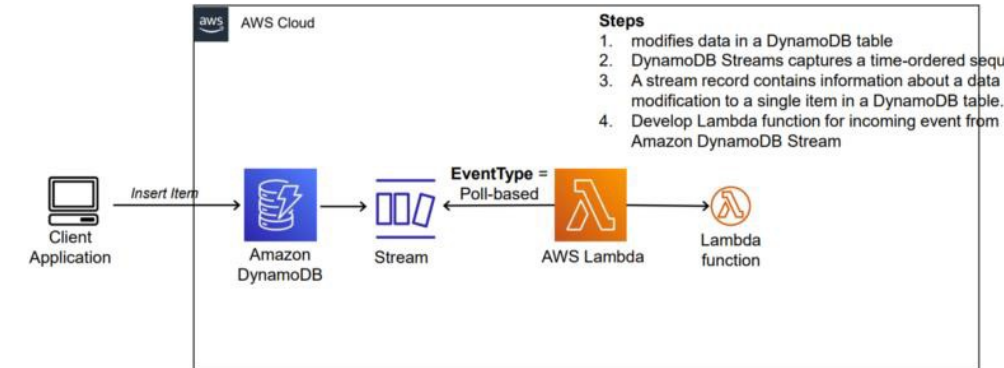
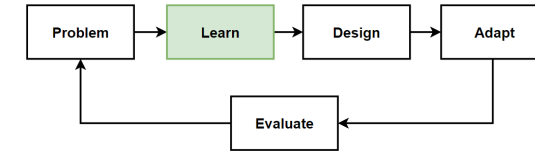
- **Azure Cosmos DB** is a **globally distributed, multi-model** database service offered by **Microsoft Azure**.
- It supports various **database models**, including document, **key-value**, **column-family**, and **graph**, and can be accessed through multiple APIs, such as **SQL**, **MongoDB**, **Cassandra**, and **Azure Table Storage**.
- **Azure Cosmos DB** has built-in support for **Change Data Capture (CDC)**, allows it to **track and propagate data changes** made to the **database** to **other systems** in near real-time.
- **CDC in Azure Cosmos DB** works by **continuously monitoring the transaction logs** of the **database for changes** and **extracting those changes** to be sent to target systems.
- **Azure Cosmos DB** change **feed API** to access the data changes and process them in your application.



<https://learn.microsoft.com/en-us/azure/cosmos-db/nosql/change-feed-design-patterns>

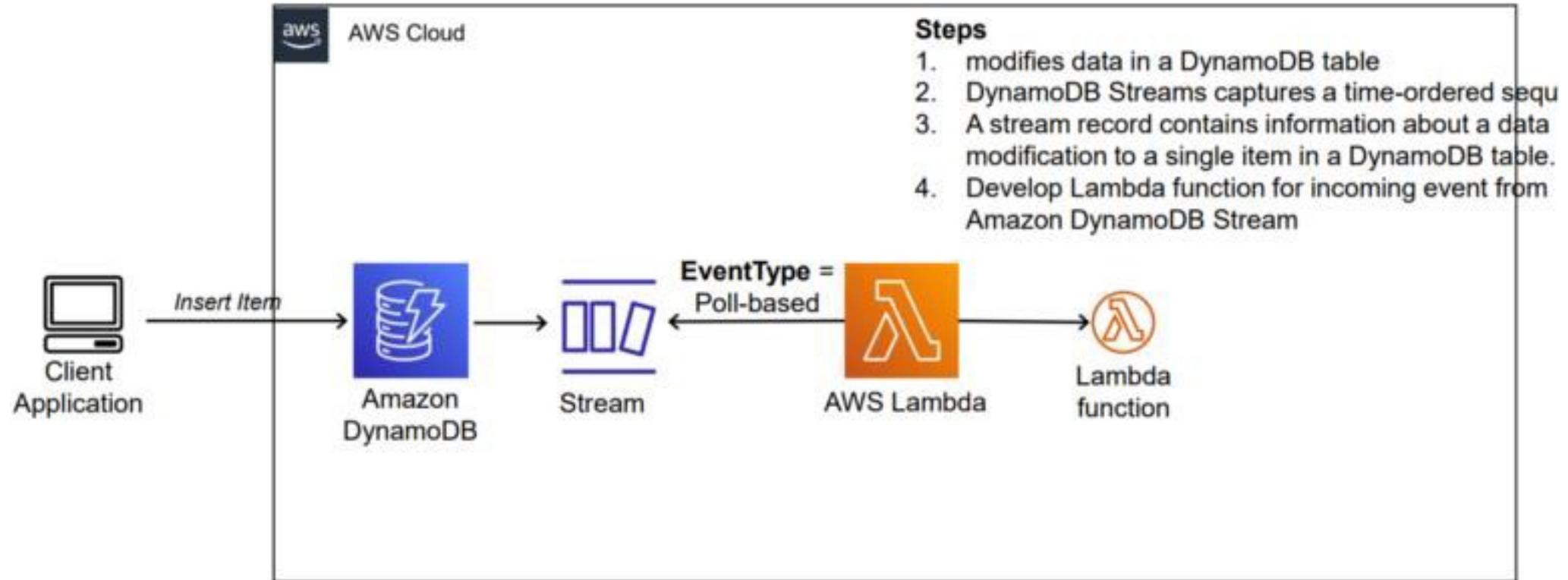
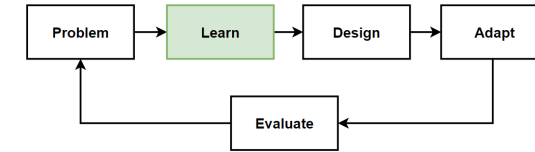
Amazon DynamoDB Streams for CDC and Outbox

- **DynamoDB** supports **streaming** of **item-level change data capture** records in **the near-real time**.
- **DynamoDB stream** is an **ordered flow of information** about changes to items in a **DynamoDB table**.
- Whenever an application **creates, updates, or deletes items** in the **table**, **DynamoDB Streams writes a stream record** with the **primary key attributes** of the items that were modified.
- A **stream record** contains information about a **data modification** to a **single item** in a **DynamoDB table**. That can includes capture additional information, such as the “**before**” and “**after**” **images** of modified items.
- **DynamoDB Streams** writes **stream records** in near-real time so that you can build applications that **consume these streams** and **take action** based on the contents.
- Most of the applications can **benefit** from **data capturing changes** into DynamoDB table; **Notifications, Mobile Apps, Financial Apps**.



<https://medium.com/aws-lambda-serverless-developer-guide-with-hands/dynamodb-streams-using-aws-lambda-to-process-dynamodb-streams-for-change-data-capture-2e3ab8df27ca>

Amazon DynamoDB Streams for CDC and Outbox



<https://medium.com/aws-lambda-serverless-developer-guide-with-hands/dynamodb-streams-using-aws-lambda-to-process-dynamodb-streams-for-change-data-capture-2e3ab8df27ca>

Before Design – What we have in our design toolbox ? - Data

Architectures Patterns&Principles		Microservices Data Choosing Database	Microservices Data Commands&Queries	FR
• Microservices Architecture	• The Database-per-Service Pattern	• The Shared Database Anti-pattern	• Materialized View Pattern	• List products
	• Polygot Persistence			• Filter products as per brand and categories
	• Decompose services by scalability	• Relational and NoSQL Databases	• CQRS Design Pattern	• Put products into the shopping cart
	• The Scale Cube	• CAP Theorem–Consistency, Availability, Partition Tolerance	• Event Sourcing Pattern	• Apply coupon for discounts
	• Microservices Decomposition Pattern	• Data Partitioning: Horizontal, Vertical and Functional Data Partitioning	• Eventual Consistency Principle	• Checkout the shopping cart and create an order
	• Microservices Communications Patterns	• Database Sharding Pattern		• List my old orders and order items history
	• Microservices Data Management Patterns			Non-FR
				• High Scalability
				• High Availability
				• Millions of Concurrent User
				• Independent

Before Design – What we have in our design toolbox ? - New

Architectures Patterns&Principles

- **Microservices Architecture**
 - The Database-per-Service Pattern
 - Polygot Persistence
 - Decompose services by scalability
 - The Scale Cube
 - Microservices Decomposition Pattern
 - Microservices Communications Patterns
 - Microservices Data Management Patterns
 - **Microservices Distributed Transaction Pattern**

Microservices Data Choosing Database

- The Shared Database Anti-pattern, Relational and NoSQL Databases
- CAP Theorem—Consistency, Availability, Partition Tolerance
- Data Partitioning: Horizontal, Vertical and Functional Data Partitioning
- Database Sharding Pattern

Microservices Data Commands&Queries

- Materialized View Pattern
- CQRS Design Pattern
- Event Sourcing Pattern
- Eventual Consistency

Microservices Distributed Transactions

- **SAGA Pattern**
- **Choreography and Orchestration-based SAGA**
- **Compensating Transaction Pattern**
- **Dual-Write Problem**
- **Transactional Outbox Pattern**
- **CDC - Change Data Capture**

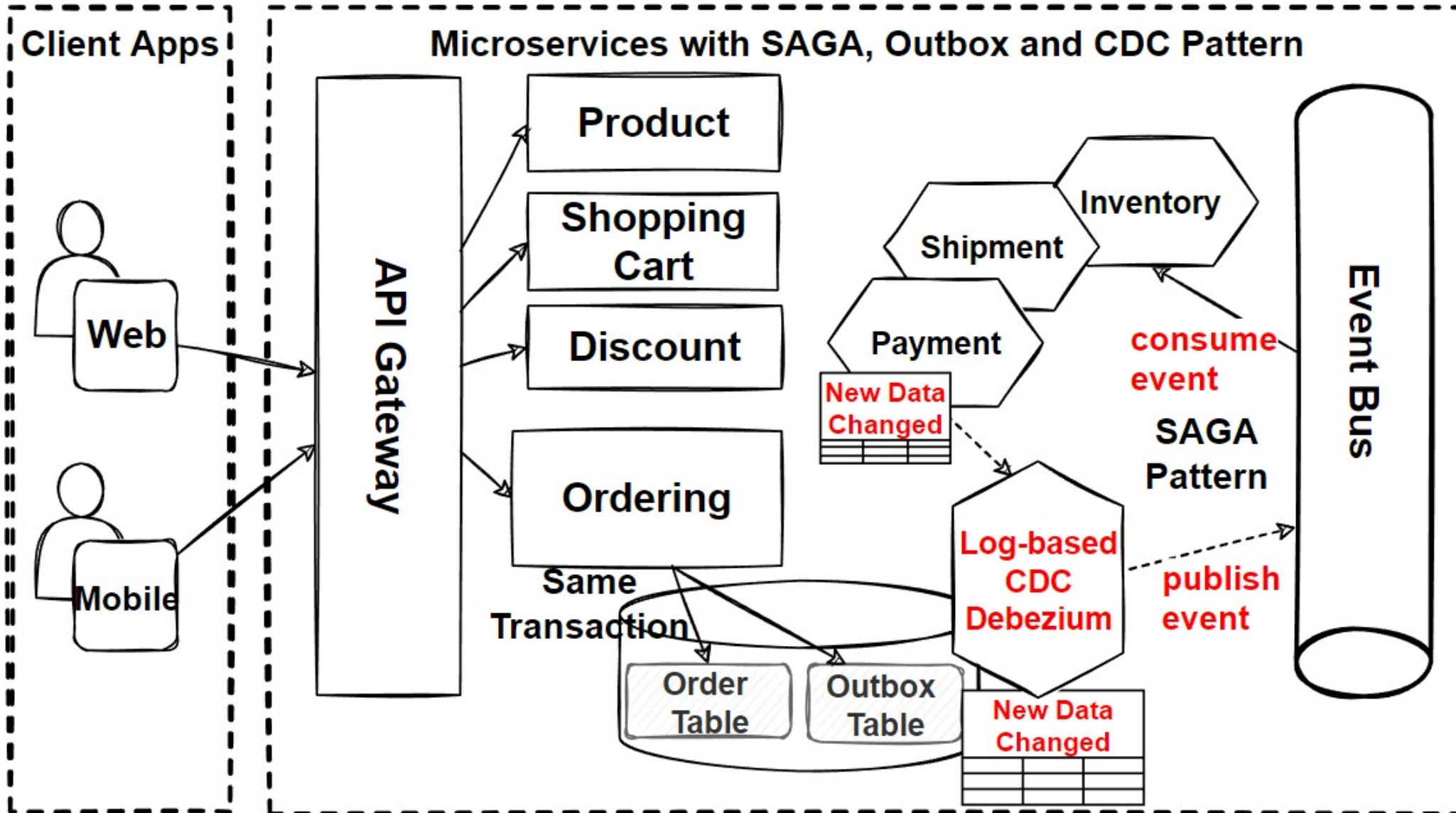
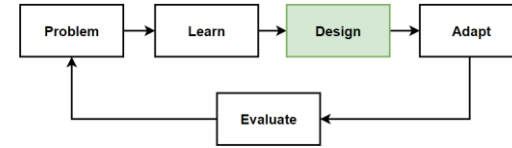
FR

- List products
- Filter products as per brand and categories
- Put products into the shopping cart
- Apply coupon for discounts
- Checkout the shopping cart and create an order
- List my old orders and order items history

Non-FR

- High Scalability
- High Availability
- Millions of Concurrent User
- Independent

Microservices with SAGA, Transactional Outbox and CDC Pattern



Adapt: Microservice Architecture with SAGA, Transactional Outbox and CDC Pattern

Frontend SPAs

- Angular
- Vue
- React

API Gateways

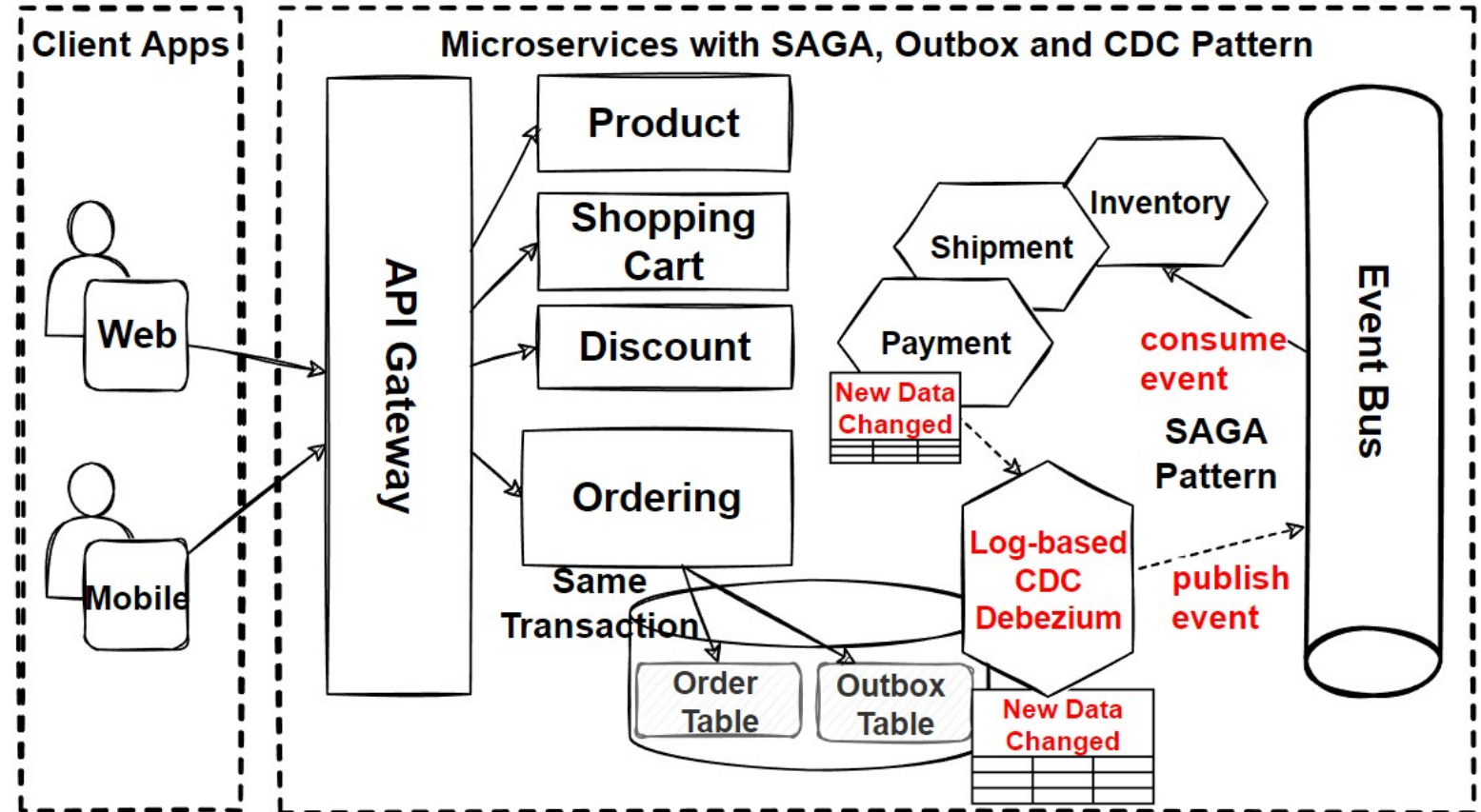
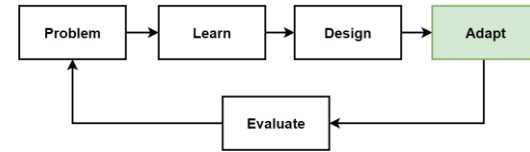
- Kong Gateway
- Express Gateway
- Amazon AWS API Gateway

Message Brokers Event Bus

- **Kafka**
- RabbitMQ
- Amazon EventBridge, SNS

Backend Microservices

- Java – Spring Boot
- .Net – Asp.net
- JS – NodeJS



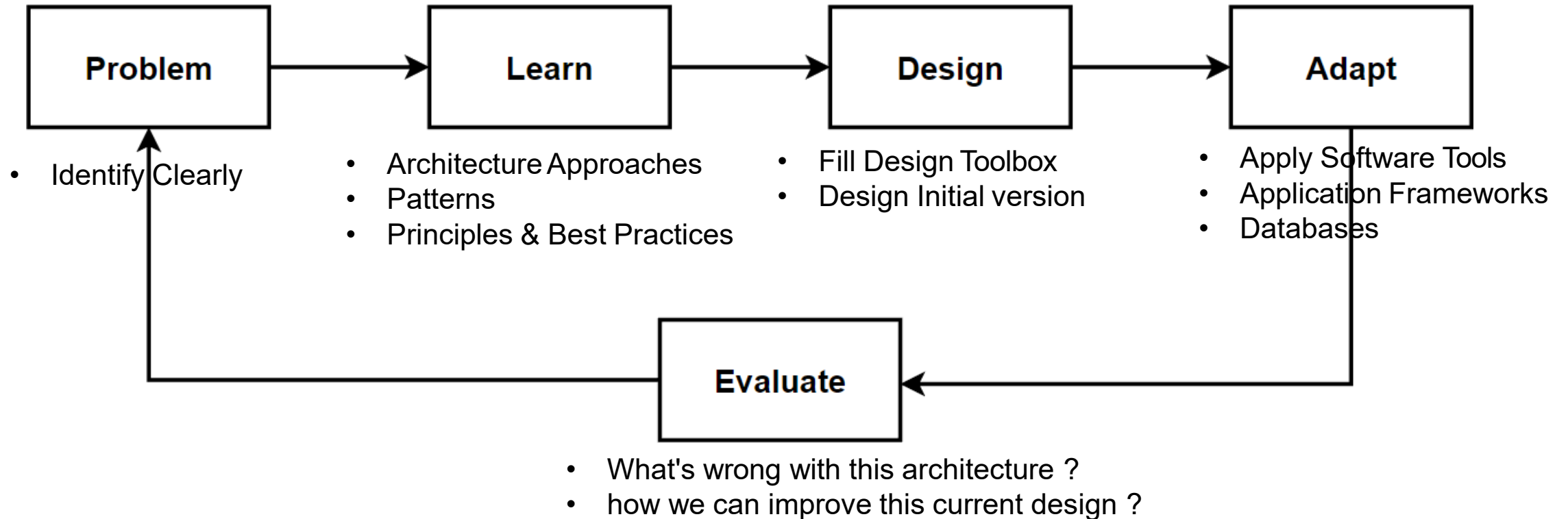
Log-based built-in CDC Database

- CockroachDB
- Azure CosmosDB
- Amazon DynamoDB Streams

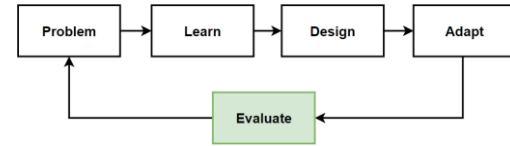
CDC – Open Source Tool

- Debezium

Way of Learning – The Course Flow



Evaluate: Microservice Architecture with SAGA, Transactional Outbox and CDC Pattern

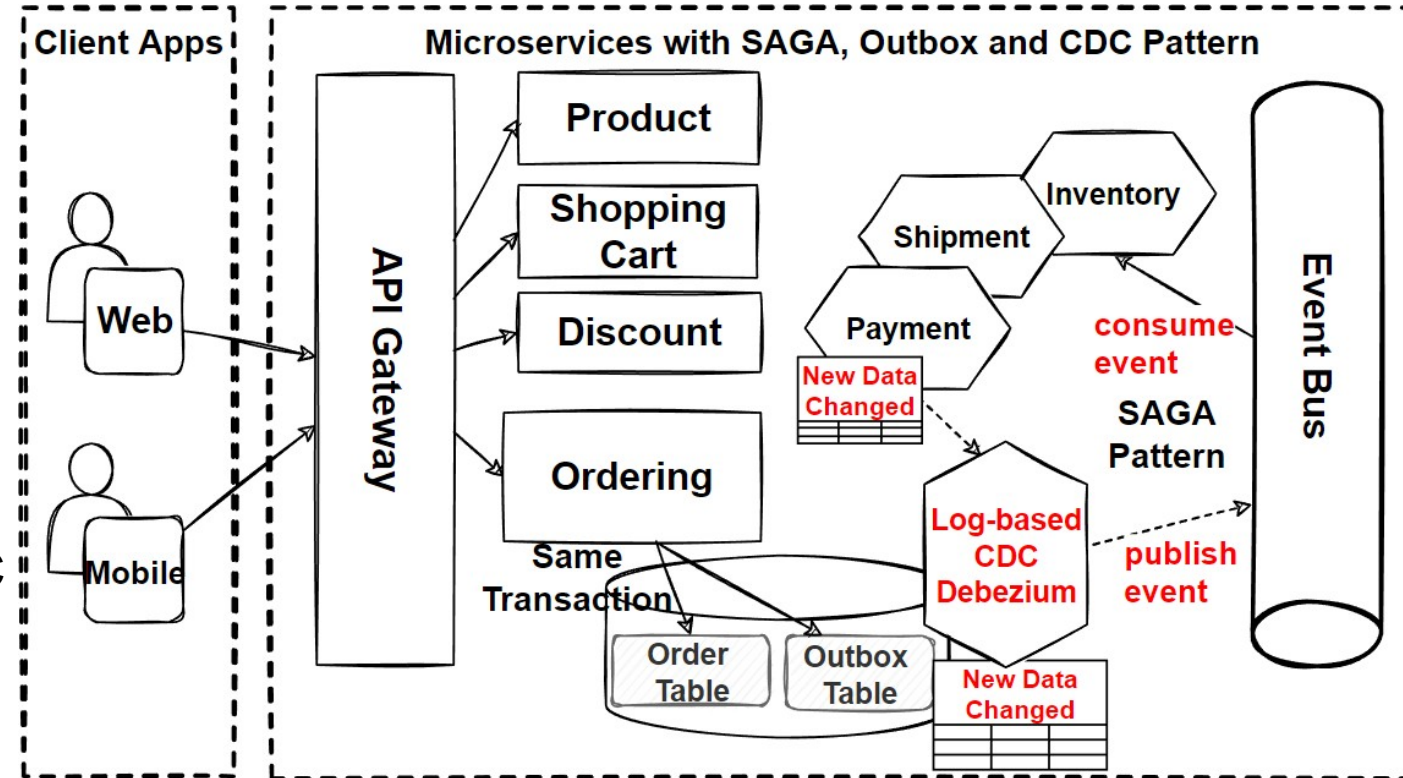


Benefits

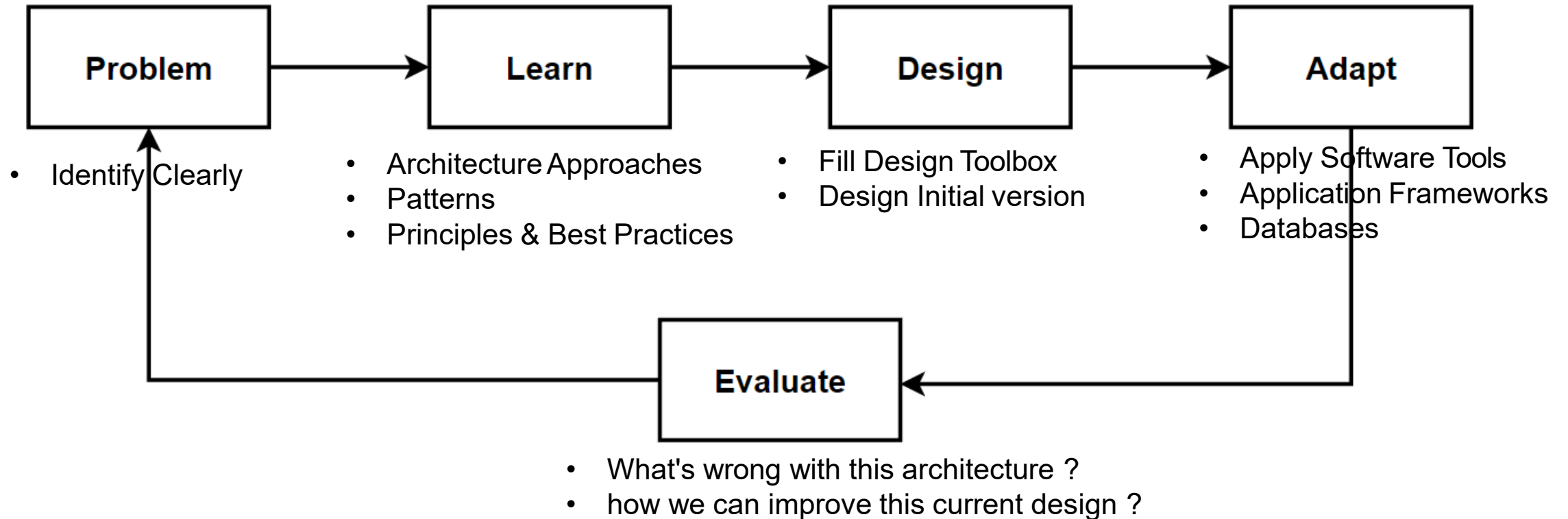
- Maintain the integrity of your database transactions
- Real-time synchronization
- Extracting and propagating changes to be kept up-to-date with the data
- Decouple data and propagate events
- Easier to scale and maintain a microservices

Drawbacks

- Increased Complexity, Outbox Pattern and CDC makes your system more complex design.
- Add latency to the event publishing process
- Performance burden on the database
- Difficult to set up and maintain



Way of Learning – The Course Flow



Problem: Handle Millions of Events Across Microservices

Considerations

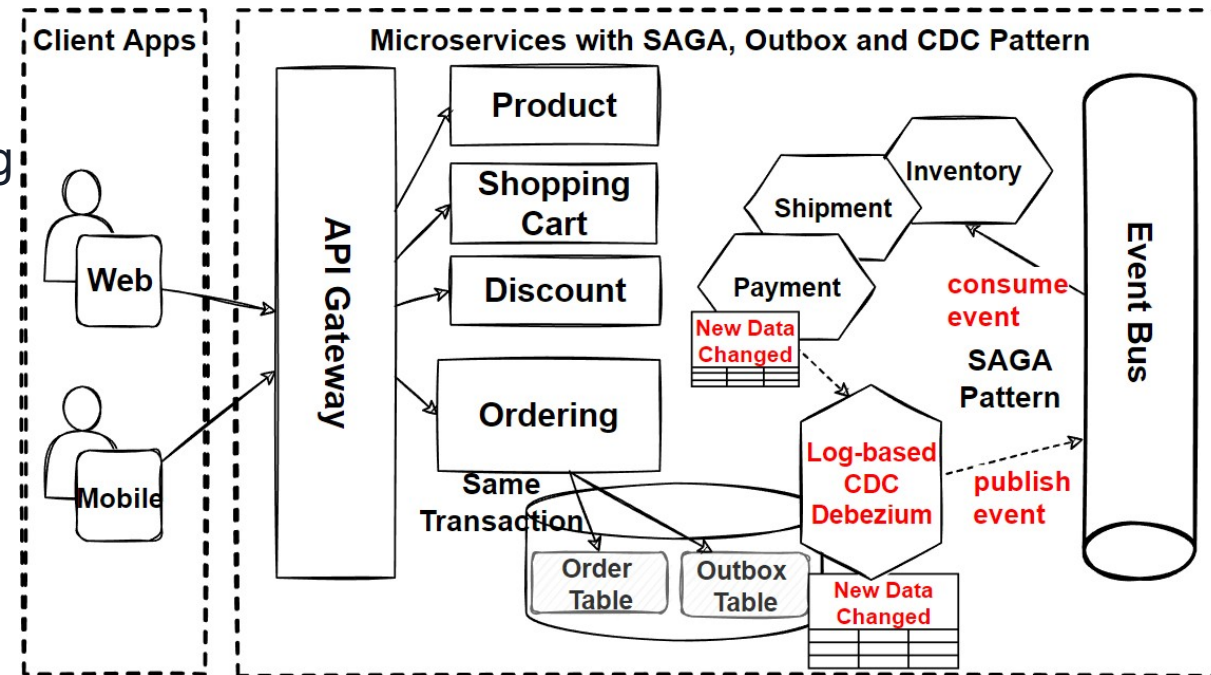
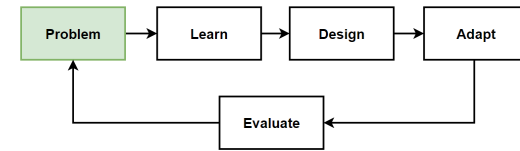
- What if we have thousands of microservices that need to communicate with millions of events ?
- If multiple subsystems must process the same events
- Required Real-time processing with minimum latency.
- Required complex event processing, like pattern matching
- Required process high volume and high velocity of data, i.e. IoT apps.

Problems

- Decoupled communications for thousands of microservices
- Real-time processing
- Handle High volume events

Solutions

- Event-driven architecture for microservices



**CÁM ƠN ĐÃ CHÚ Ý
LẮNG NGHE!**